



UNIVERSITÀ DI PISA

DIPARTIMENTO DI MATEMATICA

Corso di Laurea Triennale in Matematica

Tensor Canonical Polyadic Decomposition and the Complexity of Matrix Multiplication

Tesi di Laurea Triennale

Relatore:

Prof. Leonardo Robol

Candidato:

Alberto Defendi

ANNO ACCADEMICO 2025/2026

SOMMARIO

La decomposizione CP (Canonical Polyadic Decomposition) di rango r di un tensore è la sua scomposizione nella somma di r tensori di rango uno, una generalizzazione naturale della decomposizione di matrici. Oltre ad avere un'importanza teorica nel calcolo tensoriale, è uno strumento utile in molte aree scientifiche: per citarne alcune, spettroscopia, *signal processing*, neurologia e analisi dei dati. In molte di queste applicazioni l'unicità della fattorizzazione è una proprietà fondamentale per rappresentare fedelmente le componenti di sistemi complessi—si pensi al problema di modellare i neuroni di un sistema nervoso tramite un tensore per studiarne gli impulsi, noto come *blind source separation*.

Il primo obiettivo è dimostrare l'unicità della fattorizzazione CP sotto opportune ipotesi. Questo risultato, originariamente dimostrato da Kruskal nel 1977 attraverso il concetto di Kruskal rank, è stato di recente semplificato seguendo la dimostrazione originale, usando strumenti geometrici. Inoltre, osserveremo alcune peculiarità teoriche che distinguono i tensori dalle matrici, ponendo le basi per l'utilizzo del calcolo tensoriale nella teoria della complessità.

Successivamente, applicheremo questi strumenti al problema della complessità della moltiplicazione tra matrici: in quanto mappa bilineare, il prodotto matriciale può essere rappresentato tramite un tensore tre indici, e poi messo in corrispondenza con un algoritmo per valutare il prodotto in funzione delle entrate del tensore. Introduciamo la nozione di complessità aritmetica del prodotto tra matrici per definire l'esponente asintotico ω , che ne misura la complessità ottimale. Tale quantità sarà messa in corrispondenza con il rango CP di un tensore tramite il Teorema di Strassen. Come corollario, ridurre lo studio della complessità ai tensori, anziché all'analisi diretta delle equazioni algebriche, semplifica notevolmente la ricerca di algoritmi più efficienti; una sfida aperta da decenni, dall'algoritmo di Strassen (del 1970), fino alle recenti scoperte guidate dall'intelligenza artificiale.

Infine, l'analisi verrà estesa dagli algoritmi di moltiplicazione esatti agli algoritmi approssimati (APA), introdotti da Bini et al. In questo contesto emerge il fenomeno topologico del border rank, per il quale una successione di tensori di rango inferiore può convergere a un tensore di rango superiore. Dal punto di vista algoritmico, ciò si traduce nella possibilità di approssimare la moltiplicazione esatta tramite una successione di algoritmi con costo computazionale inferiore. Di conseguenza, il rango tensoriale può essere sostituito con il border rank nel calcolo di ω , permettendo di ottenere stime asintotiche più accurate, al prezzo di un errore algoritmico arbitrariamente piccolo.

INDICE

1	APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA	5
1.1	Mappe multilineari e moltiplicazione tra matrici	5
1.2	Decomposizione low-rank di forme bilineari	7
1.3	Rappresentazione low-rank del prodotto tra matrici	9
1.4	L'esponente della moltiplicazione tra matrici	10
1.5	Il rango come misura della complessità	13
1.6	Algoritmi di moltiplicazione veloce	19
1.7	Alcune stime sull'esponente	23
1.8	Algoritmi APA	24
1.9	Errore algoritmico	26
2	APPLICAZIONI MODELLISTICHE DELLA DECOMPOSIZIONE CP	29
2.1	Spettroscopia fluorescente	29
2.2	Blind deconvolution di segnali	35
	ACRONIMI	43
	GLOSSARIO	45
	BIBLIOGRAFIA	47

INTRODUZIONE

In questo elaborato studiamo la decomposizione di un tensore di rango r come somma di r tensori di rango uno, dal nome Canonical Polyadic Decomposition (abbreviata, decomposizione CP), anche nota come PARAFAC (da “parallel factor analysis”) o CANDECOMP (da “canonical decomposition”). Dimostreremo l’unicità di una decomposizione sotto opportune ipotesi di indipendenza dei fattori, risultato noto come il Teorema di Kruskal. Per farlo, introdurremo il concetto di unicità di una decomposizione e di indipendenza dei fattori (detto k -rango). Questo risultato sarà molto rilevante nelle applicazioni, per identificare univocamente le componenti di un modello reale. Esploreremo due applicazioni modellistiche, dove sarà necessario garantire l’unicità. La prima è la spettroscopia fluorescente (dalla chemiometria) dove la decomposizione di un tensore che rappresenta una serie di misurazioni su un campione permette di recuperare la concentrazione delle sostanze chimiche. La seconda, la blind deconvolution (dall’analisi di segnali), dove la decomposizione consente di localizzare la posizione di utenti che trasmettono messaggi a delle antenne. Successivamente passeremo allo studio della complessità aritmetica della moltiplicazione tra matrici, modellando l’algoritmo di moltiplicazione tramite la decomposizione CP di rango r . Dimostreremo la corrispondenza tra il rango della decomposizione e la complessità ottima del prodotto matriciale. Per fornire stime più accurate di tale complessità, analizzeremo gli algoritmi di moltiplicazione approssimati, che richiedono un numero inferiore di moltiplicazioni al costo di un maggiore errore algoritmico.

Le decomposizioni di tensori occorrono in numerose aree scientifiche in cui è necessario modellare elementi di sistemi complessi tramite fattori semplici. In molte di queste applicazioni è cruciale garantire l’unicità. Nell’ambito medico della psicommetria, ad esempio, sono state impiegate per studiare gli impulsi cerebrali [311, 71, 158], in particolare per individuare l’area del cervello di un paziente in cui è in atto una crisi epilettica, o la diagnosi e gestione di ictus; sempre in medicina, trovano applicazione nell’interpretare le immagini di risonanza magnetica; vedi ad esempio e relative reference. Sono inoltre utilizzate nell’elaborazione di immagini per alcune

reference

tecniche di compressione, o in generale nell'analisi di dati e in discipline come la geologia. Nell'analisi numerica, infine, sono utilizzate per approssimare funzioni di n variabili tramite una somma finita di prodotti di funzioni in una variabile.

ref

reference

Questo elaborato è strutturato come segue:

Nel Capitolo ?? introdurremo i tensori e i relativi spazi. Introdurremo il rango di un tensore, e discuteremo alcune caratteristiche dei tensori che li differenziano dalle matrici, come border rank, rango tipico e rango massimo. Definiremo la fattorizzazione CP e un metodo per calcolarla, chiamato algoritmo Alternating Least Squares (ALS). La trattazione è riadattata da Ballard e Kolda [5].

Nel Capitolo ?? definiremo l'unicità di una fattorizzazione e il Kruskal rank per dimostrare il Teorema di Kruskal, che afferma l'unicità della scrittura della fattorizzazione CP di un tensore a tre indici quando viene rispettata una relazione tra il rango e k -rango. Enunceremo il risultato per tensori n -dimensionali, e osserveremo che esso non è valido per il caso delle matrici. Presenteremo una versione modernizzata, a cura di Landsberg [24], della dimostrazione originale di Kruskal [21] in dimensione tre

Nel Capitolo 1 sposteremo il focus sulla complessità della moltiplicazione matriciale attraverso gli algoritmi di moltiplicazione veloci. Introdurremo l'esponente ottimo ω della moltiplicazione e un modello computazionale per calcolarlo. Osserveremo che la moltiplicazione tra matrici vista come mappa bilineare, può essere rappresentata come un tensore, il quale verrà poi messo in corrispondenza con un algoritmo per valutare il prodotto in funzione delle entrate del tensore stesso (detto bilinear computation). Dimosteremo la corrispondenza tra il rango di un tensore, il numero di moltiplicazioni di un algoritmo e l'esponente ottimo, fornendo alcune stime su quest'ultimo. Deriveremo un esempio di algoritmo di moltiplicazione approssimato partendo da quello classico, dove potremo osservare il fenomeno del border rank. Al termine accenneremo l'errore algoritmico degli algoritmi veloci, e confronteremo numericamente l'andamento dell'errore degli algoritmi esatti rispetto a quelli approssimati. Le fonti di questo capitolo si trovano all'intersezione dei campi dell'analisi numerica [5], teoria della complessità algebrica [11] e geometria algebrica [12, 23, 24].

Nel Capitolo 2 illustreremo alcune applicazioni modellistiche della CP attraverso due esperimenti numerici: il recupero della concentrazione di sostanze chimiche in un composto (spettroscopia fluorescente) e la ricostruzione di segnali (cocktail party problem). In entrambe le applicazioni l'unicità avrà un ruolo fondamentale.

cite

Come evidenziato da Landsberg, nell'ambito del calcolo tensoriale avviene uno

“scontro di culture”, dove confluiscono risultati teorici trovati dagli studiosi delle applicazioni scientifiche (e.g., psicometria, chemiometria, neurologia), analisti numerici, informatici, teorici della complessità e geometri algebrici. Poiché in questi settori le notazioni differiscono in modo sostanziale, abbiamo cercato di mantenere il punto di vista dell’analisi numerica, senza appesantire troppo con la notazione tipica dell’algebra multilineare.

Le sperimentazioni dei Capitoli ?? e 2 sono state sviluppate in Python con le librerie Numpy, Scipy, Tensorly, il codice sorgente si trova QUI. L’esperimento del Capitolo 3 è stato scritto in Rust, ed è parte di un progetto svolto durante il corso di laurea per il corso “laboratorio computazionale” QUI REPO.

Molte figure sono state ispirate dal libro di Ballard e Kolda [5] e riprodotte tramite Tikz. I grafici sono stati generati con Matplotlib con la libreria Scienceplots [18].

1

APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA

Aggiornato

L'algoritmo di moltiplicazione matriciale si può esprimere come mappa bilineare che manda due matrici A e B nel prodotto $C = AB$. Possiamo naturalmente rappresentare tale mappa come un tensore T , e quindi cercarne una rappresentazione di rango più basso. Questo dà il vantaggio di valutare le matrici A , B in un algoritmo che dipende dalle entrate di T per ridurre il numero di moltiplicazioni in base al rango di T . Infatti, studiamo gli *algoritmi di moltiplicazione veloce*, ovvero gli algoritmi di moltiplicazione matriciale che impiegano asintoticamente un numero minore di operazioni aritmetiche rispetto all'algoritmo classico, ottenendo un esponente di complessità asintotica minore di 3 per le matrici quadrate. In questo contesto, mostriamo che il rango della forma bilineare dà il numero di moltiplicazioni di un algoritmo, e viceversa. Risulta utile tener traccia del costo ottimale della moltiplicazione ω e dimostrare attraverso il Teorema di Strassen che il rango del tensore può essere messo in corrispondenza con ω . Concludiamo definendo l'algoritmo di Strassen e ricavando un algoritmo approssimato a partire da quello classico, sviluppando la loro fattorizzazione tensoriale e studiando i rispettivi errori algoritmici.

1.1 MAPPE MULTILINEARI E MOLTIPLICAZIONE TRA MATRICI

Definizione 1.1.1 (Moltiplicazione matriciale). Fissati degli interi positivi m, n, p , la moltiplicazione tra due matrici $m \times n$ e $n \times p$ è una mappa bilineare

$$\langle m, n, p \rangle: \mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \rightarrow \mathbb{C}^{m \times p}$$
$$(A, B) \mapsto C = AB.$$

Spesso ci riferiremo a $\langle m, n, p \rangle$ chiamandolo “algoritmo”, ad esempio nel contesto degli algoritmi di moltiplicazione veloce.

Osservazione 1.1.2. La bilinearità della mappa sopra segue immediatamente dalla definizione del prodotto matriciale:

$$(1.1) \quad (AB)_{ij} = \sum_{k=1}^n a_{ik}b_{kj},$$

dove $i = 1, \dots, m$ e $j = 1, \dots, p$.

Osservazione 1.1.3. Possiamo vedere la moltiplicazione tra matrici anche come una forma trilineare

$$\mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \times \mathbb{C}^{m \times p} \rightarrow \mathbb{C}$$

$$(A, B, C) \mapsto \text{trace}(ABC)$$

RAPPRESENTAZIONE IN COORDINATE DI UNA FORMA BILINEARE

Consideriamo degli spazi vettoriali U, V, W di dimensioni I, J, K , rispettivamente. Un modo semplice di rappresentare una forma bilineare $\varphi: U \times V \rightarrow W$ come un tensore è farlo partendo dalle slices. Fissato $k \in \{1, \dots, K\}$ e considerata la sua componente k -esima $\varphi_k: U \times V \rightarrow \mathbb{R}$, esiste una matrice $M \in \mathbb{R}^{I \times J}$ tale che $\varphi_k(u, v) = u^\top M v$ per ogni $(u, v) \in \mathbb{R}^I \times \mathbb{R}^J$. Scrivendo ogni componente di

$\varphi = \begin{bmatrix} \varphi_1 \\ \vdots \\ \varphi_K \end{bmatrix}$ in questa forma, otteniamo un tensore $T \in \mathbb{R}^{I \times J \times K}$ tale che le sue fette frontali $T_{:, :, 1}, \dots, T_{:, :, K}$ rappresentano ognuna delle K forme bilineari φ_k , ossia che

$$(1.2) \quad \varphi_k(u, v) = u^\top T_{:, :, k} v = \sum_{i=1}^I \sum_{j=1}^J T_{i,j,k} u_i v_j, \text{ per ogni } k = 1, \dots, K.$$

In forma tensoriale,

$$(1.3) \quad \varphi(u, v) = \begin{bmatrix} \varphi_1(u, v) \\ \vdots \\ \varphi_K(u, v) \end{bmatrix} = \begin{bmatrix} u^\top T_{:, :, 1} v \\ \vdots \\ u^\top T_{:, :, K} v \end{bmatrix} = T \times_1 u^\top \times_2 v^\top.$$

Osservazione 1.1.4. L'algebra multilineare garantisce l'esistenza e l'unicità di T .

Osservazione 1.1.5. Questa procedura si estende anche al caso in cui le entrate di u e v siano oggetti algebrici più complessi di scalari, ad esempio vettori o matrici.

Ricordando che $U^* \otimes V^* \otimes W$ è l'insieme delle forme trilineari da $U \times V \times W^* \rightarrow \mathbb{K}$, possiamo dare un nome all'oggetto sopra.

Definizione 1.1.6 (Tensore strutturale). L'unico tensore $T \in U^* \otimes V^* \otimes W$ associato alla mappa bilineare $\varphi : U \times V \rightarrow W$ è detto *tensore strutturale* di φ .

Osservazione 1.1.7. Con un abuso di notazione tipicamente usato in letteratura [11, 23, 24], nel seguito identificheremo l'algoritmo di moltiplicazione matriciale (come mappa bilineare) con il suo tensore strutturale, denotando entrambi dello stesso simbolo $\langle m, n, p \rangle$, distinguendo l'oggetto in questione laddove creerà ambiguità. Più avanti questa notazione sarà giustificata dalla Proposizione 1.2.3.

1.2 DECOMPOSIZIONE LOW-RANK DI FORME BILINEARI

In § ?? abbiamo osservato che le fattorizzazioni low-rank riducono il costo in memoria. Aggiungiamo che questa rappresentazione è anche vantaggiosa per ridurre il calcolo delle operazioni aritmetiche. Per una matrice $M \in \mathbb{R}^{m \times n}$ di rango uno, possiamo scrivere $M = u \otimes v = uv^\top$ per certi $u \in \mathbb{R}^m$ e $v \in \mathbb{R}^n$; la valutazione $Mx = (uv^\top)x = u(v^\top x)$ ha costo $O(n)$, contro il costo $O(mn)$ del prodotto classico matrice vettore. In generale, se M ha rango R , possiamo scriverla come somma di R matrici di rango uno e per linearità ottenere un costo $O(Rn)$ per Mx . È possibile generalizzare quest'idea per mappe bilineari.

Calcolare la valutazione di una mappa bilineare φ come nell'Equazione (1.2) senza alcuna fattorizzazione di T ha un costo $O(mnp)$, data dal numero di moltiplicazioni necessarie fra gli elementi di u e v con le entrate non nulle di T . Osserveremo che queste operazioni si chiameranno *active multiplications*, in quanto saranno legate alla misura di complessità $L(\varphi)$. In particolare, sarà possibile ridurre le *active multiplications* trovando fattorizzazioni di rango più basso di T .

Definizione 1.2.1 (Bilinear algorithm). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali. Per ogni $i \in \{1, \dots, r\}$, siano $f_i \in U^*, g_i \in V^*, w_i \in W$ tali che

$$\varphi(u, v) = \sum_{i=1}^r f_i(u)g_i(v)w_i,$$

per ogni $u \in U, v \in V$. Allora la r -upla $(f_1, g_1, w_1; \dots; f_r, g_r, w_r)$ viene detta *bilinear computation (algorithm) di lunghezza r per φ* . La lunghezza di più breve algoritmo di calcolo per φ è chiamata *bilinear complexity* o rango di φ , che indichiamo ancora con $\mathbf{R}(\varphi)$.

La prossima proposizione assicura che il rango di una forma bilineare sia dato dal rango CP.

Proposizione 1.2.2. *Fissiamo una forma bilineare φ e denotiamo con T il tensore che la rappresenta. Allora, dalla fattorizzazione CP di un tensore T si ottiene una bilinear computation, e in particolare $\mathbf{R}(\varphi) = \mathbf{R}(T)$.*

Dimostrazione. Data una decomposizione $T = \llbracket U, V, W \rrbracket$ di rango r , dalla Definizione ?? abbiamo che $T = \sum_{l=1}^r u_l \otimes v_l \otimes w_l$, dove $U \in \mathbb{R}^{I \times r}$, $V \in \mathbb{R}^{J \times r}$, $W \in \mathbb{R}^{K \times r}$. Ricordando che in componenti $T_{i,j,k} = \sum_{l=1}^r u_{il} v_{jl} w_{kl}$, la relazione dell'Equazione (1.2) diventa,

$$\begin{aligned}
 \varphi_k(x, y) &= \sum_{i=1}^I \sum_{j=1}^J T_{ijk} x_i y_j \\
 &= \sum_{i=1}^I \sum_{j=1}^J \sum_{l=1}^r u_{il} v_{jl} w_{kl} x_i y_j \\
 (1.4) \quad &= \sum_{l=1}^r \left(\sum_{i=1}^I u_{il} x_i \right) \cdot \left(\sum_{j=1}^J v_{jl} y_j \right) w_{kl} \\
 &= \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_{kl},
 \end{aligned}$$

per ogni $k = 1, \dots, K$. Mettendo insieme le Equazioni (1.3) e (1.4) troviamo

$$(1.5) \quad \varphi(x, y) = \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_l.$$

Considerando i funzionali $f_l(x) := u_l^\top x$ e $g_l(y) := v_l^\top y$ indotti dal prodotto scalare standard si ha la tesi. □

Quindi valutare φ richiede esattamente $r = \mathbf{R}(T)$ active multiplications (e $O(r)$ addizioni), che sopra evidenziamo con $(\cdot)$. In particolare, il seguente risultato algebrico ci dice che la rappresentazione di una forma bilineare come tensore o come bilinear computation sono identiche. La seconda forma sarà utile per moltiplicare le matrici sfruttando la decomposizione di T .

Proposizione 1.2.3 (Si veda [11], Cap. 14). *Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali. Allora esiste un unico isomorfismo $U^* \otimes V^* \otimes W \rightarrow \{\varphi : U \times V \rightarrow W, \text{ bilineare}\}$ che manda $s \otimes t \otimes w$ nella mappa bilineare $(u, v) \mapsto f(u)g(v)w$.*

Osservazione 1.2.4. Grazie a questo isomorfismo, possiamo passare da una bilinear computation

$$\varphi(u, v) = \sum_{i=1}^r f_i(u) g_i(v) w_i,$$

al suo tensore

$$T = \sum_{i=1}^r f_i \otimes g_i \otimes w_i,$$

e viceversa.

1.3 RAPPRESENTAZIONE LOW-RANK DEL PRODOTTO TRA MATRICI

Possiamo applicare il risultato sopra per valutare il prodotto tra matrici con un numero ottimale di operazioni.

Consideriamo due matrici $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times p}$ e il loro prodotto $C \in \mathbb{R}^{m \times p}$. Ponendo $x = \text{vec}(A)$ e $y = \text{vec}(B)$, l'Equazione (1.3) diventa

$$(1.6) \quad \text{vec}(C^\top) = \varphi(x, y) = T \times_1 \text{vec}(A)^\top \times_2 \text{vec}(B)^\top,$$

dove tramite le trasposizioni abbiamo fissato l'ordinamento naturale sulle entrate di $\text{vec}(A)$ e $\text{vec}(B)$, e quello inverso su $\text{vec}(C^\top)$. Per costruzione, il tensore $T \in \mathbb{R}^{mn \times np \times mp}$ rappresenta l'algoritmo $\langle m, n, p \rangle$. Scrivendo T tramite la sua decomposizione CP di rango r , l'Equazione (1.5) ci permette di riscrivere l'espressione sopra come una bilinear computation

$$(1.7) \quad \text{vec}(C^\top) = \sum_{l=1}^r (u_l^\top \text{vec}(A)) \cdot (v_l^\top \text{vec}(B)) w_l = \sum_{l=1}^r (f_l \cdot g_l) w_l = \sum_{l=1}^r m_l w_l,$$

Dove abbiamo posto $f_l := f_l(A) := u_l^\top \text{vec}(A)$, $g_l := g_l(B) := v_l^\top \text{vec}(B)$ e $m_l := f_l g_l$.

Questa formula definisce un algoritmo di moltiplicazione veloce $\langle m, n, p \rangle$; spesso nella pratica m, n, p sono più piccoli delle matrici da moltiplicare; in questo caso è possibile dividerle a blocchi e applicare l'algoritmo in maniera ricorsiva, ripetendo la ricorsione fino a raggiungere il caso base. Di seguito giustificheremo meglio questa procedura.

1.4 L'ESPONENTE DELLA MOLTIPLICAZIONE TRA MATRICI

In questo capitolo useremo il seguente modello computazionale con lo scopo di calcolare la complessità di valutare una forma bilineare, che semplifica il calcolo dei costi eliminando le condizioni di branching (e.g. **if-then-else** o **while**) e consentendo l'input da un alfabeto infinito di simboli (come un campo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$).

Definizione 1.4.1 (Circuito aritmetico). Un *circuito aritmetico* Γ su $\mathbb{K}$ è un grafo diretto, aciclico, finito con vertici di grado entrante 0 o 2 ed esattamente un vertice di grado uscente 0. I vertici di grado entrante 0 sono etichettati con gli elementi di $\mathbb{K} \cup \{x_1, \dots, x_n\}$ e sono detti *inputs*. Quelli di grado entrante 2 sono etichettati con $+$ oppure $\times$ e sono chiamati *gates*. Se il grado uscente di un vertice v è 0, allora v è chiamato un *output gate*. La dimensione di Γ è il numero di archi.

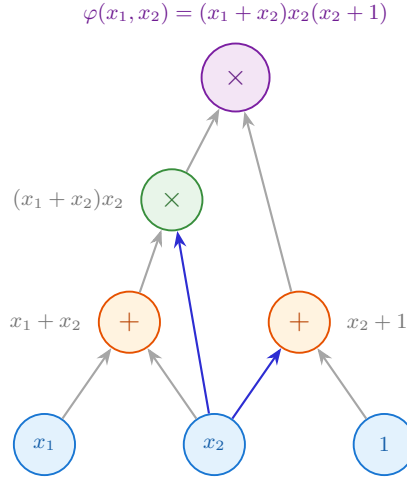


Figura 1.1: Esempio di circuito aritmetico per il polinomio $\varphi(x_1, x_2) = (x_1 + x_2)x_2(x_2 + 1)$.

Definizione 1.4.2 (Funzione costo). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali di dimensione finita. Definiamo la *funzione costo* associata a un circuito aritmetico Γ come

$$C_\Gamma(\varphi) := \text{numero di moltiplicazioni per calcolare } \varphi \text{ sul circuito } \Gamma, \text{ oppure}$$

$$C_\Gamma^{\text{tot}}(\varphi) := \text{numero di addizioni e moltiplicazioni per calcolare } \varphi \text{ sul circuito } \Gamma.$$

Denotiamo con $\mathcal{C}_\varphi$ come l'insieme dei circuiti aritmetici Γ che calcolano φ .

Definizione 1.4.3 (Funzione di complessità aritmetica). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali di dimensione finita. Definiamo

$$L(\varphi) := \inf_{\Gamma \in \mathcal{C}_\varphi} \{C_\Gamma(\varphi)\} \quad \text{la complessità moltiplicativa di } \varphi, \text{ e}$$

$$L^{\text{tot}}(\varphi) := \inf_{\Gamma \in \mathcal{C}_\varphi} \{C_\Gamma^{\text{tot}}(\varphi)\} \quad \text{la complessità totale di } \varphi.$$

Osservazione 1.4.4. In altre parole, $L(\varphi)$ è il minimo numero di moltiplicazioni non scalari sufficienti a calcolare φ , anche dette *active multiplications*. In maniera analoga, L^{tot} è il minimo numero di moltiplicazioni non scalari e di addizioni sufficienti a calcolare φ .

Consideriamo la forma bilineare $\varphi = \langle n, n, n \rangle$ data dall'Equazione (1.1). Fissato un campo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, e delle matrici $A, B \in \mathbb{R}^{n \times n}$, denotiamo con

$$(1.8) \quad M_{\mathbb{K}}(n) := L^{\text{tot}}(\langle n, n, n \rangle) = L^{\text{tot}}\left(\left\{\sum_{k=1}^n a_{ik}b_{kj} \mid i, j \in \{1, \dots, n\}\right\}\right)$$

il numero totale di moltiplicazioni aritmetiche sufficienti a calcolare la moltiplicazione di due matrici $n \times n$. Determinare una formula per $M_{\mathbb{K}}(n)$ è lontano dagli strumenti attualmente disponibili. Invece, il problema di approssimare la crescita asintotica della funzione $n \mapsto M_{\mathbb{K}}(n)$ è fattibile; definiamo

Definizione 1.4.5. L'esponente $\omega_{\mathbb{K}}$ di una moltiplicazione matriciale è il numero

$$\omega_{\mathbb{K}} := \inf_{n \in \mathbb{N}} \{\tau \in \mathbb{R} \mid \text{la moltiplicazione tra due matrici in } \mathbb{K}^{n \times n} \\ \text{ha costo } o(n^\tau) \text{ operazioni aritmetiche}\}.$$

In altre parole, $\omega_{\mathbb{K}} = \inf_{n \in \mathbb{N}} \{\tau \in \mathbb{R} \mid M_{\mathbb{K}}(n) = o(n^\tau)\}$.

Osservazione 1.4.6. L'esponente $\omega_{\mathbb{K}}$ rappresenta l'esponente ottimo globale per moltiplicare due matrici. Osserviamo che non dipende dalla taglia n .

Osservazione 1.4.7. La notazione $M_{\mathbb{K}}(h)$ e $\omega_{\mathbb{K}}$ è dovuta alla dipendenza dal campo $\mathbb{K}$. Dato che tratteremo solo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, spesso scriveremo ω per $\omega_{\mathbb{K}}$.

Aggiunto

Esempio 1.4.8. Per le matrici 2×2 , vale $4 \leq M_{\mathbb{K}}(2) \leq 2^2(4-1) = 12$. Osserveremo che trovare un algoritmo che riduce $M_{\mathbb{K}}(2)$ migliora anche l'esponente.

Teorema 1.4.9 (Schönhage [27]). *L'esponente della moltiplicazione tra matrici è invariante rispetto alle estensioni scalari: se $\mathbb{K} \subset \mathbb{L}$ è un'estensione di campo, allora*

$$\omega_{\mathbb{K}} = \omega_{\mathbb{L}}$$

STIMARE L'ESPONENTE

Un modo elementare di trovare una nuova stima su ω è trovare una scrittura più breve della forma bilineare (o del tensore). La seguente proposizione ci fornisce una prima stima della misura di complessità con il rango di una forma bilineare.

Proposizione 1.4.10 (Strassen, si veda [11], Capitolo 14). *Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali di dimensione finita, e $\varphi : U \times V \rightarrow W$ una mappa bilineare. Allora*

$$L(\varphi) = \text{Lunghezza della più breve bilinear computation per } \varphi.$$

Osservazione 1.4.11. Da questa proposizione segue direttamente che

$$(1.9) \quad L(\varphi) \leq \mathbf{R}(\varphi) \leq 2L(\varphi).$$

Osserviamo che $M_{\mathbb{K}}(n) \leq n^2(2n - 1)$. Siccome il costo computazionale dev'essere almeno pari al numero di entrate della matrice, vale $M_{\mathbb{K}}(n) \geq n^2$, da cui $\omega \in [2, 3]$. Trovare ω è un problema centrale nella teoria della complessità, e si congettura che $\omega = 2$, cioè che il costo delle moltiplicazioni possa essere asintoticamente pari a quello delle addizioni. L'algoritmo naive ha $\omega = 3$, mentre il primo miglioramento, dovuto all'algoritmo di Strassen [32] mostra che $\omega \leq \log_2(7) < 2.81$. Il record attuale è $\omega \leq 2.371177$ [3]. Dopo Strassen, nel 1978 i miglioramenti significativi sono stati $\omega \leq 2.7962$ ad opera di [26], poi $\omega \leq 2.7799$ [7] nel 1979, poi $\omega \leq 2.55$ [27], poi $\omega \leq 2.48$ nel 1987 [31], e poi $\omega \leq 2.38$ [12] nel 1989, che potrebbe aver fatto pensare che nel 1990 si sarebbe trovato il bound. Dopo tale miglioramento, l'approssimazione è scesa molto lentamente, fino a scendere sotto $\omega < 2.372$ solo recentemente [2, 17, 30, 33]. Tuttavia, l'avvento dell'Intelligenza Artificiale sta dando un nuovo impulso a questa ricerca (si veda ad esempio AlphaTensor [16]); di recente (agosto 2026) l'utilizzo di strumenti di IA come AlphaEvolve ha stabilito il nuovo record di $\omega \leq 2.371177$ [3]. Sottolineiamo che da un punto di vista pratico, non è detto che le stime ottimali si traducano in algoritmi più veloci o stabili di quello classico, ad esempio l'algoritmo di Strassen, che presenteremo, in molti casi compete con algoritmi con esponente più basso [6, 14].

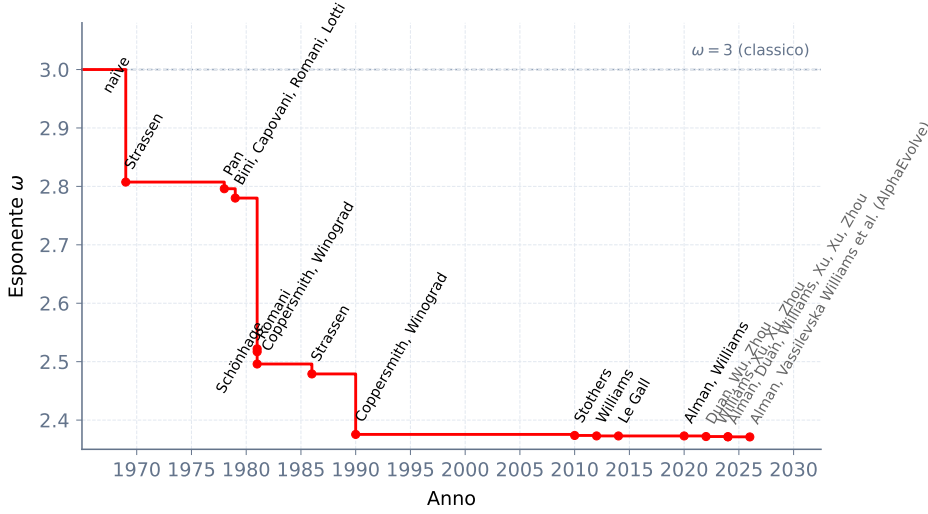


Figura 1.2: Miglioramenti del limite superiore dell'esponente della moltiplicazione matriciale ω dal 1969 ad oggi. La linea tratteggiata rappresenta il limite superiore $\omega \leq 3$.

1.5 IL RANGO COME MISURA DELLA COMPLESSITÀ

STIMARE LA COMPLESSITÀ NEL CASO GENERALE

Precedentemente abbiamo introdotto gli algoritmi di moltiplicazione dal caso base (indicandolo con $\langle n, n, n \rangle$) senza giustificare come cambia la complessità quando l'algoritmo viene eseguito ricorsivamente. Date due matrici $n^i \times n^i$, possiamo vedere la loro moltiplicazione come il prodotto di matrici $n \times n$ sull'algebra delle matrici $n^{i-1} \times n^{i-1}$, suddividendole in blocchi. Questo dice anche che, se $\mathbf{R}(\langle n, n, n \rangle) \leq r$, allora $\mathbf{R}(\langle n^i, n^i, n^i \rangle) \leq r^i$; infatti, combinando le prossime Proposizioni 1.5.4 e 1.5.5, e usando che il rango è invariante per isomorfismo, troviamo

$$\mathbf{R}(\langle n^i, n^i, n^i \rangle) = \mathbf{R}(\langle n, n, n \rangle^{\otimes i}) \leq \mathbf{R}(\langle n, n, n \rangle)^i \leq r^i,$$

dove

$$\langle n, n, n \rangle^{\otimes i} = \underbrace{\langle n, n, n \rangle \otimes \cdots \otimes \langle n, n, n \rangle}_{i\text{-volte}}.$$

In altre parole, possiamo applicare ricorsivamente l'algoritmo di moltiplicazione veloce, che è disegnato per matrici $n \times n$, per ottenere la moltiplicazione di matrici $n^i \times n^i$ impiegando solo r^i moltiplicazioni. In questo modo possiamo evitare dover fattorizzare il tensore di $\langle n^i, n^i, n^i \rangle$, che sarebbe del modesto formato $(n^i)^2 \times (n^i)^2 \times (n^i)^2$.

Se invece $n > n_0$ sono due interi positivi (nei passaggi può essere utile pensare al caso $n_0 = 2$) dato un algoritmo con un caso base fissato $\langle n_0, n_0, n_0 \rangle$ è possibile ottenerne uno per $\langle n, n, n \rangle$ con $n > n_0$, allargando la matrice di taglia $n \times n$ aggiungendo zeri fino a raggiungere la dimensione $n_0^{\lceil \log_{n_0} n \rceil}$, in modo tale che le matrici da moltiplicare siano partizionate in blocchi di taglia divisibile dal caso base. La complessità dipenderà comunque da n_0 e da r , in quanto

$$\begin{aligned} \mathbf{R}(\langle n, n, n \rangle) &\leq \mathbf{R}(\langle n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil} \rangle) \quad \text{dalla 1.5.3,} \\ &\leq \mathbf{R}(\langle n_0, n_0, n_0 \rangle)^{\lceil \log_{n_0} n \rceil} \quad \text{dalle 1.5.4 e 1.5.5,} \\ &\leq r^{\lceil \log_{n_0} n \rceil} \quad \text{per ipotesi } \mathbf{R}(\langle n, n, n \rangle) \leq r, \\ &\leq r \cdot n^{\log_{n_0} r} \quad \text{usando che } \lceil x \rceil \leq x + 1. \end{aligned}$$

Riassumendo, $\mathbf{R}(\langle n, n, n \rangle) \leq r \cdot n^{\log_{n_0} r}$. Dunque ignorando la costante e applicando l'Equazione (1.15), segue che $\omega \leq \log_{n_0} r$. Abbiamo dimostrato la seguente:

Proposizione 1.5.1. *Se $\mathbf{R}(\langle n, n, n \rangle) \leq r$ per degli interi positivi n, r , allora $n^\omega \leq r$.*

Osservazione 1.5.2. Questo risultato tornerà utile per stimare l'esponente a partire dal tensore strutturale di un algoritmo di moltiplicazione veloce (un modo alternativo per stimarlo è applicare il Master Theorem [13]).

Nei passaggi sopra abbiamo anche osservato che se $n \leq n'$ e dato un algoritmo per n' , possiamo ottenerne uno per n allargando la matrice. Possiamo racchiudere questa proprietà nel seguente risultato:

Proposizione 1.5.3 (Monotonia del rango di un algoritmo). *Il rango di un algoritmo per la moltiplicazione di matrici è monotono crescente rispetto alle loro taglie, ovvero dati due interi positivi $n \leq n'$, allora $\mathbf{R}(\langle n, n, n \rangle) \leq \mathbf{R}(\langle n', n', n' \rangle)$.*

Riportiamo le proposizioni utilizzate per dimostrare la proposizione sopra, dimostrate in [11, Capitoli 14 e 15].

Proposizione 1.5.4. *Siano e, e', h, h', l, l' degli interi positivi. Allora abbiamo*

$$\langle e, h, l \rangle \otimes \langle e', h', l' \rangle \simeq \langle ee', hh', ll' \rangle,$$

cioè le due forme bilineari sono isomorfe.

Proposizione 1.5.5. *Siano φ_1, φ_2 due mappe bilineari. Allora $\mathbf{R}(\varphi_1 \otimes \varphi_2) \leq \mathbf{R}(\varphi_1)\mathbf{R}(\varphi_2)$.*

Dimostrazione. Scriviamo φ_1, φ_2 come bilinear computation:

$$\begin{aligned}\varphi_1(u_1, v_1) &= \sum_{i=1}^{r_1} \alpha_i(u_1) \beta_i(v_1) w_i, \\ \varphi_2(u_2, v_2) &= \sum_{j=1}^{r_2} \gamma_j(u_2) \beta_j(v_2) z_j.\end{aligned}$$

Calcolando il loro prodotto tensoriale,

$$\begin{aligned}(\varphi_1 \otimes \varphi_2)(u_1 \otimes u_2, v_1 \otimes v_2) &= \varphi_1(u_1, v_1) \otimes \varphi_2(u_2, v_2) \\ &= \left(\sum_{i=1}^{r_1} \alpha_i(u_1) \beta_i(v_1) w_i \right) \otimes \left(\sum_{j=1}^{r_2} \gamma_j(u_2) \beta_j(v_2) z_j \right) \\ &= \sum_{i=1}^{r_1} \sum_{j=1}^{r_2} \alpha_i(u_1) \beta_i(v_1) \gamma_j(u_2) \beta_j(v_2) (w_i \otimes z_j),\end{aligned}$$

che è un'espressione di $\varphi_1 \otimes \varphi_2$ come somma di mappe bilineari di rango uno. Per minimalità del rango, segue

$$\mathbf{R}(\varphi_1 \otimes \varphi_2) \leq r_1 r_2 = \mathbf{R}(\varphi_1) \mathbf{R}(\varphi_2).$$

□

IL TEOREMA DI STRASSEN

Dato un algoritmo con r moltiplicazioni abbiamo mostrato che è possibile costruire la sua bilinear computation (e rappresentarla con il suo circuito aritmetico) e poi dedurre che il rango del tensore è r . Con la Proposizione 1.5.1 osservato che, trovato un tensore di rango r riduce il costo computazionale. A questo punto è lecito chiedersi se, dato l'insieme di tutti gli algoritmi di moltiplicazione $\langle n, n, n \rangle$ al variare di n , trovo una stima ottimale su ω .

Teorema 1.5.6 (Strassen [32], vedi anche [11] §15.1).

$$\omega = \inf_n \{ \tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau) \}.$$

Oververo, è possibile moltiplicare due matrici $n \times n$ usando $O(n^{\omega+\varepsilon})$ operazioni aritmetiche se e solo se il rango tensoriale di $\langle n, n, n \rangle$ è $O(n^{\omega+\varepsilon})$.

Introduciamo alcuni strumenti utili a dimostrare questo risultato.

non mi soddisfa!

non capisco come sia collegata alla prop 1.5.1

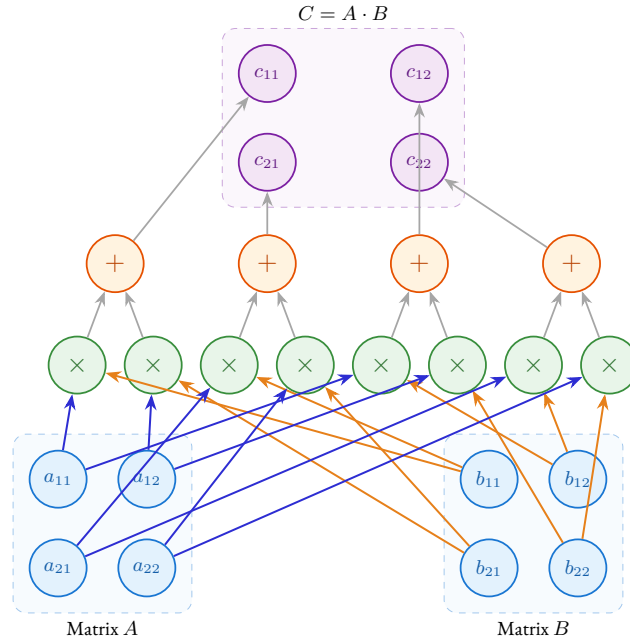


Figura 1.3: Circuito aritmetico per la moltiplicazione di matrici 2×2 .

Definizione 1.5.7 (Mappa bilineare concisa). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare. Diremo che φ è *concise* se valgono tutte e tre le condizioni sotto:

1. $\text{Ker}_L(\varphi) = \{u \in U \mid \varphi(u, \cdot) = 0\} = \{0\}$,
2. $\text{Ker}_R(\varphi) = \{v \in V \mid \varphi(\cdot, v) = 0\} = \{0\}$,
3. $\text{Im}(\varphi) = \{\varphi(u, v) \mid u \in U, v \in V\} = W$.

Osservazione 1.5.8. Vedendo φ come un elemento di $U^* \otimes V^* \otimes W$ è possibile legare questa definizione alla Definizione ?? per i tensori concise.

Esempio 1.5.9. La mappa bilineare $\langle n, n, n \rangle$ è concise.

Lemma 1.5.10. Sia $x = (x_0, x_1, x_2, \dots)$ una successione di numeri reali che soddisfa la ricorsione $x_s = ax_{s-1} + cb^s$, per certi numeri reali a, b, c . Allora per ogni $s \geq 0$, vale

$$x_s = \begin{cases} a^s x_0 + (a^s - b^s) \frac{bc}{a-b} & \text{se } a \neq b, \\ (cs + x_0) a^s & \text{se } a = b. \end{cases}$$

change to roman and to i-concise per matchare quella dei tensori

Dimostrazione. Mostriamo la formula per induzione su s . Per $s = 0$ segue direttamente:

$$\begin{aligned} x_1 &= ax_0 + bc, \\ x_2 &= a(ax_0 + bc) + b^2c \\ &= a^2x_0 + (a+b)bc \\ &= \begin{cases} a^2x_0 + (a^2 - b^2)\frac{bc}{a-b} & \text{se } a \neq b, \\ (2c + x_0)a^2 & \text{se } a = b. \end{cases} \end{aligned}$$

Assumiamo la proprietà vera per $n - 1$ e mostriamola per n .

Se $a \neq b$,

$$\begin{aligned} x_s &= a \left(a^{s-1}x_0 + (a^{s-1} - b^{s-1})\frac{bc}{a-b} \right) + b^s c \\ &= a^s x_0 + ((a^{s-1} - b^{s-1})a + b^{s-1}(a-b))\frac{bc}{a-b} \\ &= a^s x_0 + (a^s - b^s)\frac{bc}{a-b} \end{aligned}$$

Similmente, se $a = b$

$$\begin{aligned} x_s &= (c(s-1) + x_0 + c)a^s \\ &= (cs + x_0)a^s. \end{aligned}$$

□

Dimostrazione del Teorema di Strassen. Dimostriamo che

$$\omega = \inf\{\tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Sia θ il lato destro dell'equazione sopra, e per semplicità scriviamo $M(n) := M_{\mathbb{K}}(n)$ e $\omega := \omega_{\mathbb{K}}$. Dall'Equazione (1.9) abbiamo

$$\frac{1}{2}R(\langle n, n, n \rangle) \leq L(\langle n, n, n \rangle) \leq M(n),$$

e dalla Definizione 1.4.5 segue $\theta \leq \omega$.

Mostriamo che $\omega \leq \theta$. Per definizione di θ , per ogni $\varepsilon > 0$ esiste un $n > 1$ tale che

$$(1.10) \quad r := \mathbf{R}(\langle n, n, n \rangle) \leq n^{\theta+\varepsilon}.$$

Dall'Equazione (1.7) esistono dei funzionali $f_\rho, g_\rho \in (\mathbb{K}^{n \times n})^*$ e delle matrici $w_\rho \in \mathbb{K}^{n \times n}$ per $\rho \in \{1, \dots, r\}$ tali che, per ogni $A, B \in \mathbb{K}^{n \times n}$

$$AB = \sum_{\rho=1}^r f_\rho(A)g_\rho(B)w_\rho.$$

Vedendo $\mathcal{A} = \mathbb{K}^{n^i \times n^i}$ come l'algebra delle matrici $n^i \times n^i$, notiamo che i funzionali $f_\rho, g_\rho : \mathbb{K}^{n \times n} \rightarrow \mathbb{K}$ si estendono a funzionali $f_\rho, g_\rho : \mathcal{A}^{n \times n} \rightarrow \mathcal{A}$, sostituendo le entrate scalari con delle matrici. In particolare vale, per ogni $A, B \in \mathcal{A}^{n \times n}$

$$(1.11) \quad AB = \sum_{\rho=1}^r f_\rho(A)g_\rho(B)w_\rho.$$

L'Equazione sopra rappresenta l'algoritmo di moltiplicazione a blocchi tra A e B : infatti abbiamo $\mathcal{A}^{n \times n} = (\mathbb{K}^{n^i \times n^i})^{n \times n} \simeq \mathbb{K}^{n^{i+1} \times n^{i+1}}$, dove l'isomorfismo può essere pensato come la suddivisione di matrici più grandi in matrici a blocchi. Da questo fatto, unito all'Equazione (1.11), si ottiene che

$$(1.12) \quad M(n^{i+1}) \leq rM(n^i) + cn^{2i},$$

dove $c := c(n, r)$ è una costante che dipende sia da n che da r . Questo perché nella formula il prodotto AB , sviluppato come prodotto tra blocchi $n^i \times n^i$ (formalmente, visto come prododotto a coefficienti in $\mathcal{A}$), coinvolge r prodotti e un numero costante di addizioni. Siccome $\langle n, n, n \rangle$ è concise, abbiamo che $r \geq n^2$. Fissando n (e quindi r), risolviamo la ricorsione dell'Equazione (1.12) in funzione di i usando il Lemma 1.5.10 con $a = r$ e $b = n^2$, analizzando due casi.

Se $r > n^2$, abbiamo che

$$\begin{aligned} M(n^i) &= r^i M(1) + (r^i - n^{2i}) \frac{n^2 c}{r - n^2} \\ &= (M(1) + n^2 c) r^i - n^{2i} \frac{n^2 c}{r - n^2}, \end{aligned}$$

da cui,

$$M(n^i) \leq \alpha r^i + \beta n^{2i},$$

dove abbiamo posto $\alpha := M(1) + n^2 c$ e $\beta := -\frac{n^2 c}{r - n^2}$.

Se $r = n^2$, dal Lemma (1.12) abbiamo

$$M(n^i) \leq (cs + M(1))r^i,$$

In entrambi i casi troviamo,

$$(1.13) \quad M(n^i) = O(r^i).$$

Dalla proposizione 1.5.3 segue che M è una successione monotona, per ogni $h \in \mathbb{N} \setminus \{0\}$,

$$(1.14) \quad M(h) \leq M(n^{\lceil \log_n h \rceil}) = O(r^{\log_n h}) = O(h^{\log_n r}) = O(h^{\theta+\varepsilon}),$$

dove la prima uguaglianza segue dall'Equazione 1.5.3, la seconda da una proprietà del logaritmo, e la terza applicando la stima dell'Equazione (1.10). Passando all'inf troviamo $\omega_{\mathbb{K}} \leq \theta$, che completa la dimostrazione. $\square$

Osservazione 1.5.11. Riassumendo, il teorema sopra mette in corrispondenza $\mathbf{R}(\langle n, n, n \rangle)$ (e quindi il numero di active multiplications) con l'esponente asintotico ottimale. Possiamo quindi scrivere

$$(1.15) \quad \omega = \lim_{n \rightarrow +\infty} \log_n(\mathbf{R}(\langle n, n, n \rangle)).$$

1.6 ALGORITMI DI MOLTIPLICAZIONE VELOCE

Osservazione 1.6.1. Di seguito ci limiteremo a studiare prodotti fra matrici quadrate; per il caso rettangolare, si possono estendere i ragionamenti sopra considerando una rappresentazione di rango r del tensore $\langle m, n, p \rangle$ e scrivendo, attraverso una serie di permutazioni cicliche, la fattorizzazione di $\langle mnp, mnp, mnp \rangle$ di rango r^3 , che produce un algoritmo di complessità $O(n^{\log_{abc} r^3}) = O(n^{3 \log_{abc} r})$. Questo si può vedere tramite la scrittura della moltiplicazione tra matrici come forma trilineare dell'Osservazione 1.1.3, che ci permette di dire che il tensore è invariante per trasformazioni cicliche,

$$(1.16) \quad \text{trace}(ABC) = \text{trace}(CAB) = \text{trace}(BAC).$$

IL CASO NAIVE

Trattiamo il caso di matrici quadrate con taglie potenze di due (possiamo farlo a meno di passare a una sottomatrice di taglia pari). In questo caso possiamo partizionare le matrici in delle sottomatrici a blocchi 2×2 ¹:

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}.$$

Qui è indifferente pensare alle entrate come scalari o come matrici. L'algoritmo naive procede combinando un insieme di otto moltiplicazioni matriciali con quattro addizioni:

$$(1.17) \quad \begin{aligned} m_1 &= a_{11} \cdot b_{11} = s_1 t_1 & m_2 &= a_{12} \cdot b_{21} = s_2 t_2 & m_3 &= a_{11} \cdot b_{12} = s_3 t_3 \\ m_4 &= a_{12} \cdot b_{22} = s_4 t_4 & m_5 &= a_{21} \cdot b_{11} = s_5 t_5 & m_6 &= a_{22} \cdot b_{21} = s_6 t_6 \\ m_7 &= a_{21} \cdot b_{12} = s_7 t_7 & m_8 &= a_{22} \cdot b_{22} = s_8 t_8 \end{aligned}$$

$$\begin{aligned} c_{11} &= m_1 + m_2 = a_{11}b_{11} + a_{12}b_{21}, & c_{12} &= m_3 + m_4 = a_{11}b_{12} + a_{12}b_{22}, \\ c_{21} &= m_5 + m_6 = a_{21}b_{11} + a_{22}b_{21}, & c_{22} &= m_7 + m_8 = a_{21}b_{12} + a_{22}b_{22}. \end{aligned}$$

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^2 a_{ik}b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j}$$

Figura 1.4: Rappresentazione della moltiplicazione di due matrici 2×2 .

Proviamo a scriverlo in forma tensoriale. La sua bilinear computation è

$$C = \sum_{i=1}^8 m_i w_i.$$

¹possiamo pensare alle entrate sia come scalari, che come matrici.

Resta solo da assegnare i termini w_i , che in questo caso saranno matrici. Fissiamo la base standard $\{E_{11}, E_{12}, E_{21}, E_{22}\}$ dello spazio delle matrici 2×2 . Scriviamo

$$\begin{aligned} C &= c_{11}E_{11} + c_{12}E_{12} + c_{21}E_{21} + c_{22}E_{22} \\ &= (m_1 + m_2)E_{11} + (m_3 + m_4)E_{12} + (m_5 + m_6)E_{21} + (m_7 + m_8)E_{22} \\ &= \sum_{i=1}^8 m_i w_i, \end{aligned}$$

dove i termini w_i sono determinati dalla scrittura sopra.

Tramite l'isomorfismo della Proposizione 1.2.3, possiamo mappare ogni termine $s_r t_r w_r$ dell'equazione sopra in $s_r \otimes t_r \otimes w_r$ e scrivere il tensore strutturale di $\langle 2, 2, 2 \rangle$ in formato CP (si veda anche la Figura 1.5). Per farlo, identifichiamo i funzionali s_r e t_r con i loro primali a_{ij} e b_{ij} e le matrici w_r con gli elementi c_{ij} .

$$\begin{aligned} \langle 2, 2, 2 \rangle &= a_{11} \otimes b_{11} \otimes c_{11} + a_{12} \otimes b_{21} \otimes c_{11} \\ &\quad + a_{21} \otimes b_{11} \otimes c_{21} + a_{22} \otimes b_{21} \otimes c_{21} \\ &\quad + a_{11} \otimes b_{12} \otimes c_{12} + a_{12} \otimes b_{22} \otimes c_{12} \\ &\quad + a_{21} \otimes b_{12} \otimes c_{22} + a_{22} \otimes b_{22} \otimes c_{22}. \end{aligned} \tag{1.18}$$

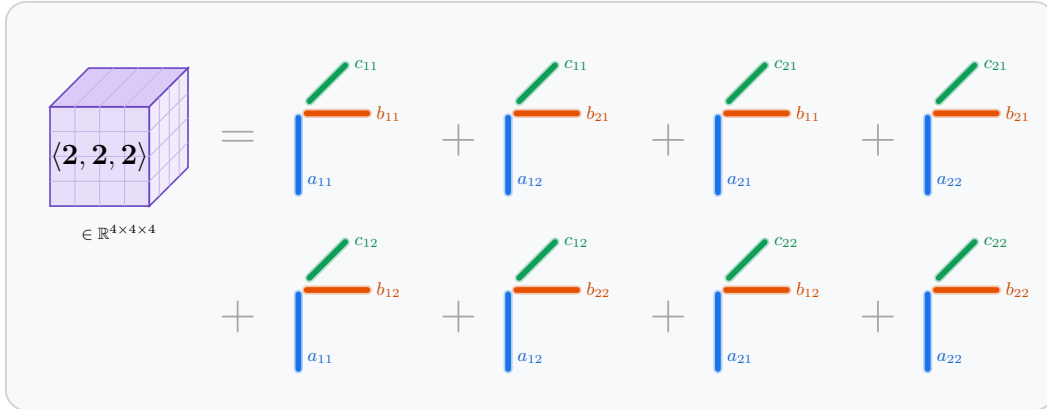


Figura 1.5: Decomposizione CP di rango 8 del tensore di moltiplicazione di matrici $\langle 2, 2, 2 \rangle$.

Osserviamo che questo caso base dà luogo a un algoritmo ricorsivo per $n > 2$. Contestualizzando l'Equazione (1.14) del Teorema di Strassen, troviamo che $M(n) \leq O(n^{\log_2 8}) = O(n^3)$.

Nel 1969 Strassen² tentò di mostrare che non esistesse un algoritmo più veloce di quello classico. Tramite una serie di riduzioni manuali riuscì a mostrare l'ottimalità su $\mathbb{Z}/2\mathbb{Z}$, ma per $\mathbb{R}$ o $\mathbb{C}$ la tesi si rivelò essere falsa.

L'ALGORITMO DI STRASSEN

L'algoritmo di Strassen è probabilmente l'algoritmo di moltiplicazione veloce più celebre e segue la stessa struttura a blocchi $\langle 2, 2, 2 \rangle$.

$$\begin{array}{lll}
 s_1 = a_{11} + a_{22} & s_2 = a_{21} + a_{22} & s_3 = a_{11} \\
 s_4 = a_{22} & s_5 = a_{11} + a_{12} & s_6 = a_{21} - a_{11} \\
 s_7 = a_{12} - a_{22} & & \\
 t_1 = b_{11} + b_{22} & t_2 = b_{11} & t_3 = b_{12} - b_{22} \\
 t_4 = b_{21} - b_{11} & t_5 = b_{22} & t_6 = b_{11} + b_{12} \\
 t_7 = b_{21} + b_{22} & &
 \end{array}$$

$$\begin{aligned}
 (1.19) \quad & m_r = s_r t_r, \quad 1 \leq r \leq 7 \\
 & c_{11} = m_1 + m_4 - m_5 + m_7 \quad c_{12} = m_3 + m_5 \\
 & c_{21} = m_2 + m_4 \quad c_{22} = m_1 - m_2 + m_3 + m_6
 \end{aligned}$$

Nella base canonica, la matrice C si scrive come

$$\begin{aligned}
 C &= c_{11}E_{11} + c_{12}E_{12} + c_{21}E_{21} + c_{22}E_{22} \\
 &= (m_1 + m_4 - m_5 + m_7)E_{11} + (m_3 + m_5)E_{12} \\
 &\quad + (m_2 + m_4)E_{21} + (m_1 - m_2 + m_3 + m_6)E_{22} \\
 &= m_1(E_{11} + E_{22}) + m_2(E_{21} - E_{22}) + m_3(E_{12} + E_{22}) \\
 &\quad + m_4(E_{11} + E_{21}) + m_5(-E_{11} + E_{12}) + m_6E_{22} + m_7E_{11} \\
 &= \sum_{r=1}^7 m_r w_r.
 \end{aligned}$$

²in una comunicazione riportata da Landsberg in [24].

Possiamo anche calcolare gli m_r :

$$\begin{aligned}
m_1 &= (a_{11} + a_{22})(b_{11} + b_{22}) \\
m_2 &= (a_{21} + a_{22})b_{11} \\
m_3 &= a_{11}(b_{12} - b_{22}) \\
m_4 &= a_{22}(b_{21} - b_{11}) \\
m_5 &= (a_{11} + a_{12})b_{22} \\
m_6 &= (a_{21} - a_{11})(b_{11} + b_{12}) \\
m_7 &= (a_{12} - a_{22})(b_{21} + b_{22})
\end{aligned}$$

Passando all'isomorfismo, e usando l'identificazione fatta sopra, la bilinear computation si può riscrivere in forma tensoriale come

$$\begin{aligned}
\langle 2, 2, 2 \rangle &= (a_{11} + a_{22}) \otimes (b_{11} + b_{22}) \otimes (c_{11} + c_{22}) \\
&+ (a_{21} + a_{22}) \otimes b_{11} \otimes (c_{21} - c_{22}) \\
&+ a_{11} \otimes (b_{12} - b_{22}) \otimes (c_{12} + c_{22}) \\
&+ a_{22} \otimes (-b_{11} + b_{21}) \otimes (c_{21} + c_{11}) \\
&+ (a_{11} + a_{12}) \otimes b_{22} \otimes (-c_{11} + c_{12}) \\
&+ (-a_{11} + a_{21}) \otimes (b_{11} + b_{12}) \otimes c_{22} \\
&+ (a_{12} - a_{22}) \otimes (b_{21} + b_{22}) \otimes c_{11}.
\end{aligned}$$

Espandendo i prodotto tensoriali e facendo opportune semplificazioni, possiamo ritrovare il tensore dell'algoritmo classico riportato nell'Equazione (1.18).

Osservazione 1.6.2 (Costo computazionale). Similmente a quanto svolto sopra, troviamo che $M(n) \leq O(n^{\log_2 7}) = O(n^{2.81})$. Inoltre, dalla Proposizione 1.5.1 troviamo che $\omega \leq \log_2 7 < 2.81$.

qui posso mettere $M(n) = O(\dots)$?

Osservazione 1.6.3. L'algoritmo di Strassen non è unico: è possibile riscrivere il tensore sopra in funzione di una costante $t \in \mathbb{R}$ arbitraria. Si veda [24, Pag. 37].

1.7 ALCUNE STIME SULL'ESPONENTE

Ad esempio, l'algoritmo di Strassen [32] mostra che $\mathbf{R}(\langle 2, 2, 2 \rangle) \leq 7$, e come dimostrato da Winograd [34], $\mathbf{R}(\langle 2, 2, 2 \rangle) = 7$, ovvero non è possibile moltiplicare due matrici 2×2 con meno di sette moltiplicazioni. Nei casi diversi da $\langle 2, 2, 2 \rangle$ il problema si complica, e attualmente abbiamo alcune stime più lasche: già per matrici

3×3 , sappiamo solo che $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$ [10, 22]; questo caso è ritenuto molto curioso perché esiste un'espressione nota per il tensore, ma è difficile calcolarne il rango. In dimensione più alta la migliore stima dal basso per matrici $n \times n$ è $\frac{5}{2}n^2 - 3n \leq \mathbf{R}(\langle n, n, n \rangle)$ [9]. Si veda [24, Capitolo 11] per una discussione più ampia.

1.8 ALGORITMI APA

Dopo il tentativo di Strassen di dimostrare che l'algoritmo di moltiplicazione standard fosse ottimale, Bini et. Al [7] considerano il seguente algoritmo di moltiplicazione ridotto ottenuto ponendo a zero l'entrata a_{22} di $\langle 2, 2, 2 \rangle$:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} \end{bmatrix}.$$

Facendo alcuni raccoglimenti e sostituendo $a_{22} = 0$, il tensore dell'Equazione (1.18) si riscrive come

$$\begin{aligned} \langle 2, 2, 2 \rangle^{\text{red}} &:= a_{11} \otimes (b_{11} \otimes c_{11} + b_{12} \otimes c_{12}) \\ &\quad + a_{12} \otimes (b_{21} \otimes c_{11} + b_{22} \otimes c_{12}) \\ &\quad + a_{21} \otimes (b_{11} \otimes c_{21} + b_{12} \otimes c_{22}). \end{aligned}$$

Che è somma di sei tensori elementari distinti, quindi $\mathbf{R}(\langle 2, 2, 2 \rangle^{\text{red}}) \leq 6$. Gli autori provarono a cercare un'espressione di rango cinque per $\langle 2, 2, 2 \rangle^{\text{red}}$ numericamente, calcolando

$$\min_{\substack{T \in \mathbb{R}^{n \times n \times n} \\ \mathbf{R}(T)=5}} \|\langle 2, 2, 2 \rangle^{\text{red}} - T\|$$

norma di $\langle 2, 2, 2 \rangle^{\text{red}}$. Il loro computer continuava a produrre tensori T di rango cinque con la norma della differenza che diventava sempre più piccola, ma con coefficienti sempre più grandi. Questo è andato avanti per un po' di tempo, fino a che realizzarono che il codice era giusto, ma quello che sembrava un errore era in realtà una manifestazione del border rank. L'espressione del tensore che stavano cercando era il seguente tensore di rango 5:

$$\langle n, n, n \rangle_t^{\text{red}} = \frac{1}{t} [(I) + (II) - (III) - (IV) + (V)],$$

dove

$$\begin{aligned}
(I) &= (a_{12} + ta_{11}) \otimes (b_{12} + tb_{22} \otimes c_{12}) \\
&= a_{12} \otimes b_{12} \otimes c_{12} + t[a_{12} \otimes b_{22} \otimes c_{12} + a_{11} \otimes b_{12} \otimes c_{12}] + O(t^2) \\
(II) &= (a_{21} + ta_{11}) \otimes b_{11} \otimes (c_{11} + tc_{21}) \\
&= a_{21} \otimes b_{11} \otimes c_{11} + t[a_{21} \otimes b_{11} \otimes c_{21} + a_{11} \otimes b_{11} \otimes c_{11}] + O(t^2) \\
(III) &= a_{12} \otimes b_{12} \otimes ((c_{11} + c_{12}) + tz_{22}) \\
&= a_{12} \otimes b_{12} \otimes c_{11} + a_{12} \otimes b_{12} \otimes c_{12} + t[a_{12} \otimes b_{12} \otimes c_{22}] \\
(IV) &= a_{21} \otimes ((b_{11} + b_{12}) + tb_{21}) \otimes c_{11} \\
&= a_{21} \otimes b_{11} \otimes c_{11} + a_{21} \otimes b_{12} \otimes c_{11} + t[a_{21} \otimes b_{21} \otimes c_{11}] \\
(V) &= (a_{12} + a_{21}) \otimes (b_{12} + tb_{21}) \otimes (c_{11} + tc_{22}) \\
&= a_{12} \otimes b_{12} \otimes c_{11} + a_{21} \otimes b_{12} \otimes c_{11} \\
&\quad + t[a_{12} \otimes (b_{12} + c_{22} + b_{21} \otimes c_{11}) + a_{21} \otimes (b_{12} \otimes c_{22} + b_{21} \otimes c_{11})] + O(t^2)
\end{aligned}$$

In ogni passaggio abbiamo applicato più volte la proprietà distributiva del prodotto tensoriale. Osserviamo che i termini di grado costante in t si cancellano:

$$\begin{aligned}
&a_{12} \otimes (b_{12} \otimes c_{12} - b_{12} \otimes c_{11} - b_{12} \otimes c_{12} + b_{12} \otimes c_{11}) \\
&+ a_{21} \otimes (b_{11} \otimes c_{11} - b_{11} \otimes c_{11} + b_{12} \otimes c_{11} - b_{12} \otimes c_{11}) = 0
\end{aligned}$$

Sommando e raggruppando i termini di grado uno in t troviamo

$$\begin{aligned}
\langle n, n, n \rangle^{\text{red}} &= a_{11} \otimes (b_{11} \otimes c_{11} + b_{12} \otimes c_{12}) + \\
&\quad a_{12} \otimes (b_{21} \otimes c_{11} + b_{22} \otimes c_{12}) + \\
&\quad a_{21} \otimes (b_{11} \otimes c_{21} + b_{12} \otimes c_{22}),
\end{aligned}$$

ovvero il tensore di partenza. Da cui,

$$\begin{aligned}
\langle n, n, n \rangle_t^{\text{red}} - \langle n, n, n \rangle^{\text{red}} &= \frac{1}{t} \left(t \langle n, n, n \rangle^{\text{red}} + O(t^2) \right) - \langle n, n, n \rangle^{\text{red}} \\
&= O(t) \xrightarrow{t \rightarrow 0} 0
\end{aligned}$$

Abbiamo dimostrato che i tensori approssimanti $\langle n, n, n \rangle_t^{\text{red}}$, di rango 5 convergono ad un tensore di rango 6. Nella pratica, è possibile calcolare il prodotto di due matrici usando il tensore $\langle n, n, n \rangle_t^{\text{red}}$ per una scelta di t maggiore della precisione di macchina, riducendo il numero di moltiplicazioni (in questo caso di una moltiplicazione), a costo

di un errore algoritmico che accenneremo in § 1.9.

Il border rank si rivela utilissimo nello studio dell'esponente ω , cioè possiamo sostituire il rango con il border rank nel Teorema 1.5.6:

Teorema 1.8.1 (Bini [7], vedi §3.2).

$$\omega = \inf\{\tau \in \mathbb{R} \mid \underline{\mathbf{R}}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Possiamo inoltre generalizzare la Proposizione 1.5.1.

Proposizione 1.8.2. Se $\underline{\mathbf{R}}(\langle n, n, n \rangle) \leq r$ per degli interi positivi n, r , allora $n^\omega \leq r$.

Anche se potremmo avere $\underline{\mathbf{R}}(\langle n, n, n \rangle) < \mathbf{R}(\langle n, n, n \rangle)$, non sono abbastanza diversi

why?

da avere un impatto sull'esponente.

ref

Riportiamo alcune stime sul border rank riportate in [23, §1]. Per $n = 2$, Winograd dimostra che $\underline{\mathbf{R}}(\langle 2, 2, 2 \rangle) = \mathbf{R}(\langle 2, 2, 2 \rangle) = 7$. Ci aspetteremmo che per $n > 2$, $\underline{\mathbf{R}}(\langle n, n, n \rangle) < \mathbf{R}(\langle n, n, n \rangle)$, ma la risposta non è così semplice. Per $n = 3$, sappiamo solo che $15 \leq \underline{\mathbf{R}}(\langle 3, 3, 3 \rangle) \leq 20$, mentre per il rango vale $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$. In generale, sappiamo che $\mathbf{R}(\langle n, n, n \rangle) \geq 3n^2 - o(n)$ e $\underline{\mathbf{R}}(\langle n, n, n \rangle) \geq 2n^2 - \lceil \log_2(n) \rceil - 1$.

1.9 ERRORE ALGORITMICO

Discutiamo brevemente l'errore algoritmico degli algoritmi di moltiplicazione veloci, rimandando a [4] per ulteriori dettagli. Gli algoritmi di moltiplicazione veloci producono errori numerici più grandi rispetto all'algoritmo classico, dove l'errore in avanti è dato componente per componente, e dipende solo dal prodotto scalare della riga e colonna corrispondente. Gli algoritmi veloci, invece, eseguono calcoli che coinvolgono altre entrate della matrice che non appaiono nel prodotto scalare (eventualmente questi contributi si cancellano), dunque gli errori dipendono da proprietà globali delle matrici di input. In gergo, gli algoritmi veloci³ sono detti stabili rispetto alla norma [8, 25], mentre quello classico gode della proprietà detta sopra, ovvero è *stabile rispetto alle componenti*, caratteristica irraggiungibile per gli algoritmi veloci [25]. Definiamo la *stabilità rispetto alla norma* per il prodotto $C = AB$ dove $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times k}$ come

$$\|\hat{C} - C\| \leq f_{\text{alg}}(n)\|A\|\|B\| + O(\varepsilon^2),$$

dove $\|\cdot\|$ è la norma del massimo, ε è la precisione di macchina, e f_{alg} è una costante polinomiale che dipende dall'algoritmo. Ad esempio, $f_{\text{alg}}(n) = n^2$ per l'algoritmo

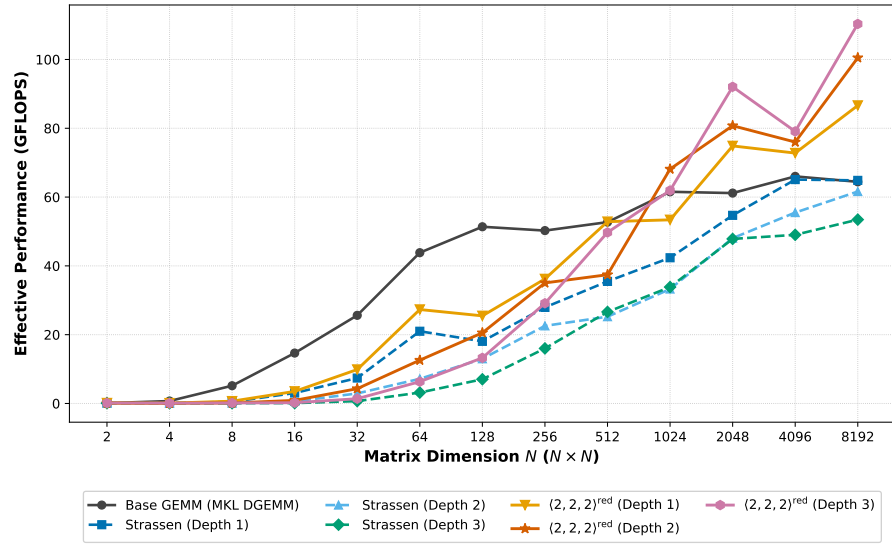
³senza ulteriori modifiche, come manipolazione algoritmica per ridurre l'errore di arrotondamento

di moltiplicazione classico, senza alcuna ipotesi sull'ordinamento dei calcoli svolti dal prodotto scalare. Osserviamo che f_{alg} è indipendente dalle matrici di input, e $\|A\|\|B\|$ è indipendente dall'algoritmo.

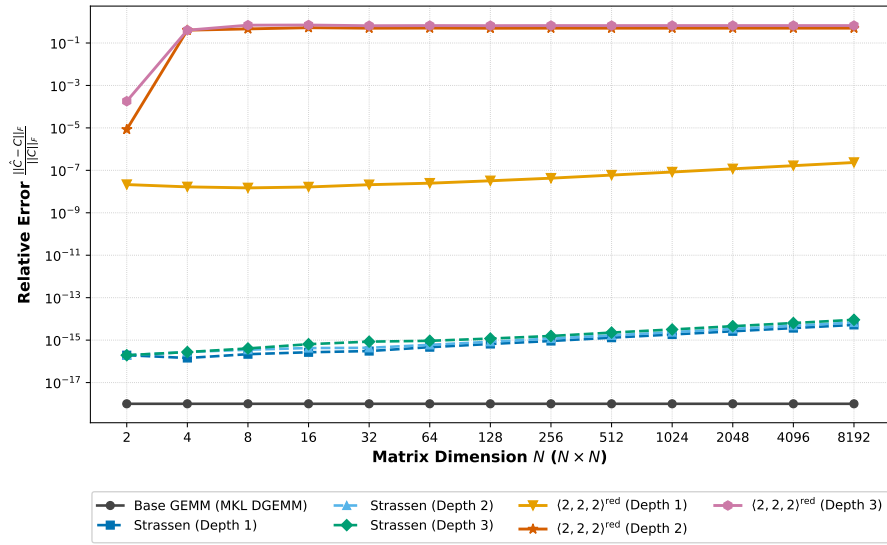
ANDAMENTO DELL'ERRORE

Per analizzare il comportamento dell'errore algoritmico abbiamo confrontato l'algoritmo $\langle 2, 2, 2 \rangle^{\text{red}}$ con l'algoritmo di Strassen e con la routine di moltiplicazione standard fornita da Intel MKL DGEMM [19] (Math Kernel Library, Double-precision General Matrix Multiply). Gli esperimenti sono stati condotti per dimensioni delle matrici $N = 2^1, \dots, 2^{13} = 8192$ e livelli di ricorsione $L \in \{1, 2, 3\}$. I risultati sono riportati in Figura 1.6.

Come mostrato in Figura 1.6a l'algoritmo $\langle 2, 2, 2 \rangle^{\text{red}}$ raggiunge prestazioni elevate grazie all'impiego di sole 5 moltiplicazioni. Tuttavia, come evidenziato in Figura 1.6b, l'accumulo degli errori di arrotondamento nel calcolo di $\hat{C}$, dovuti al parametro ε , porta ad una crescita dell'errore relativo $\frac{\|\hat{C} - C\|_F}{\|C\|_F}$, dove C è il risultato dell'algoritmo di moltiplicazione standard, all'aumentare della profondità di ricorsione ($L = 2, 3$).



(a) Prestazioni computazionali (GFLOPS effettivi)



(b) Errore relativo in norma di Frobenius

Figura 1.6: Confronto delle prestazioni computazionali (in GFLOPS effettivi) e dell'errore relativo in norma di Frobenius tra gli algoritmi al variare della dimensione della matrice $N = 2, 4, 8, \dots, 8192$.

2

APPLICAZIONI MODELLISTICHE DELLA DECOMPOSIZIONE CP

In questo capitolo esploreremo due applicazioni della decomposizione CP di un tensore a tre indici in due ambiti modellistici, dove avremo modo di applicare il Teorema di Kruskal ?? e l'algoritmo Alternating Least Squares. Nel primo esperimento il rango non sarà noto a priori e dovremo prima approssimarlo, per poi trovare una decomposizione. Nel secondo conosceremo il rango a priori. In entrambi gli esperimenti sarà fondamentale l'essenziale unicità data dal teorema di Kruskal ??. Inoltre, osserveremo che nella pratica la fattorizzazione CP soffre l'ambiguità del riscalamento e della permutazione dei fattori.

cambia titolo

why excitation-emission matrices have rank 1?

improve

2.1 SPETTROSCOPIA FLUORESCENTE

Nella *spettroscopia fluorescente*, un campione chimico è eccitato (e.g. tramite una luce), e la luce che viene emessa viene misurata a diverse lunghezze d'onda, risultando in una matrice di *eccitazione-emissione* (EEM) di intensità fluorescenti (si veda Figura 2.1). Quando i dati EEM vengono raccolti da diversi campioni, otteniamo un tensore a tre indici. Questi dati si possono utilizzare nella chimica analitica per stimare le concentrazioni e gli spettri dei composti chimici a partire da più miscele.

Nel dettaglio, vengono dati I campioni di soluzioni chimiche, ognuno dei quali contiene diverse sostanze diluite in diverse concentrazioni. Il primo obiettivo è determinare il numero r delle diverse sostanze presenti. Il dataset che analizziamo [1, 20] è composto da una serie di esperimenti riportati da [29], nel quale sono presenti ci sono tre sostanze chimiche disciolte nelle soluzioni:

- *Valine-Tyrosine-Valine* (Val-Tyr-Val) - peptide,
- *Tryptophan-Glycine* (Trp-Gly) - peptide,
- *Phenylalanine* (Phe) - amino acid.

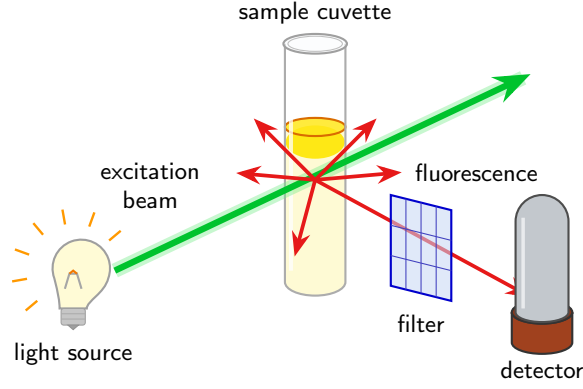


Figura 2.1: Rappresentazione schematica di uno spettrofotometro per spettroscopia di fluorescenza.

Segnaliamo che i dati sono stati preprocessati dagli autori per riempire i dati mancanti e sostituire le entrate negative, come spiegato in [20].

Ogni campione, diciamo il numero i -esimo è successivamente eccitato da una luce in J diverse lunghezze d'onda. Per ogni lunghezza d'onda di eccitazione, viene misurato lo spettro emesso. Diciamo che l'intensità della luce fluorescente emessa è misurata da K diverse lunghezze d'onda. Allora per ogni i otteniamo una matrice $J \times K$ di eccitazione-emissione.

Quindi i dati possono essere rappresentati come un array $I \times J \times K$, scrivibile come

$$T = \sum_{f=1}^r a_f \otimes b_f \otimes c_f,$$

in componenti,

$$T_{ijk} = \sum_{f=1}^r a_{if} b_{jf} c_{kf},$$

dove $a_{i,f}$ è la concentrazione della f -esima sostanza nell' i -esimo campione, $c_{k,f}$ è la frazione di protoni che la f -esima sostanza emette nella lunghezza d'onda k e $b_{j,f}$ è l'intensità della luce incidente alla lunghezza d'onda j moltiplicata per l'assorbimento alla lunghezza d'onda j . Il primo obiettivo è trovare un r tale che ogni f rappresenti una sostanza.

Il dataset analizzato contiene le misurazioni di matrici di eccitazione-emissione (EEM) per 18 campioni, ognuno dei quali consiste in una miscela dei tre composti chimici sopra (Val-Tyr-Val, Trp-Gly e Phe). I dati sono organizzati in un tensore tridimensionale di dimensioni $I \times J \times K = 18 \times 251 \times 21$, corrispondenti rispetti-

vamente ai campioni, alle lunghezze d'onda di emissione (da 250 a 500 nm) e alle lunghezze d'onda di eccitazione (da 210 a 310 nm). Una rappresentazione tridimensionale del tensore è mostrata in Figura 2.2, mentre alcune sezioni di matrici di eccitazione-emissione sono visualizzate in Figura 2.3.

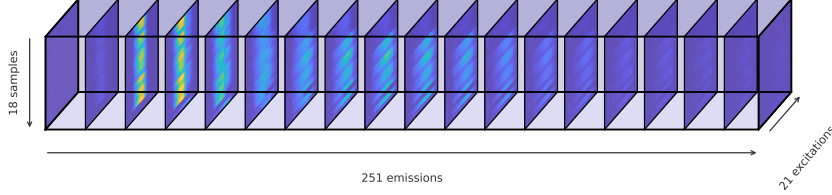


Figura 2.2: Rappresentazione tridimensionale del tensore EEM di dimensioni $18 \times 251 \times 21$.

Cerchiamo una fattorizzazione approssimata di $\hat{T}$ con le tecniche descritte nella § ???. Fissata una tolleranza ε piccola, applichiamo il procedimento dell'Algoritmo 1. La ricerca di $\hat{T}$ viene ripetuta attraverso una serie di restarts per evitare minimi locali, al termine dei quali viene restituito il tensore che minimizza l'errore relativo ε_{rel} . Nell'articolo originale [29] vengono anche esclusi i tensori che non sono ragionevoli da un punto di vista fisico per rimuovere valori di r troppo piccoli. Per i nostri scopi ci limitiamo a imporre che le quantità fisiche coinvolte (concentrazioni, intensità di emissione ed eccitazione) siano non negative, per cui si scartano le decomposizioni che presentano fattori negativi.

Algorithm 1 Algoritmo di approssimazione del tensore originale.

```

1: for  $i = 1 \dots r$  do
2:   repeat
3:      $\hat{T} \leftarrow \arg \min_{A,B,C} \|T - \llbracket A, B, C \rrbracket\|$  ▷ CP ALS con restarts.
4:   until  $\|T - \hat{T}\| < \varepsilon$ 
5: end for
```

Assumiamo ora che il rango ottimale r sia stato determinato attraverso l'euristica discussa in § ???. Siccome i valori di r sono relativamente piccoli, a meno di piccole perturbazioni, l'espressione di $\hat{T}$ come somma di elementi di rango uno sarà unica grazie al Teorema di Kruskal ???. Quindi, decomponendo $\hat{T}$, possiamo recuperare la concentrazione di ognuna delle r sostanze in ogni soluzione, determinando i vettori a_f così come lo spettro delle eccitazioni ed emissioni determinando i vettori b_f . La Figura 2.5 mostra i fattori calcolati e il confronto con le concentrazioni reali delle tre sostanze, arrestando CP ALS quando la differenza dell'errore relativo tra due iterazioni susseguenti è minore di 10^{-4} . Nel grafico sono tracciati i fattori a_f, b_f, c_f

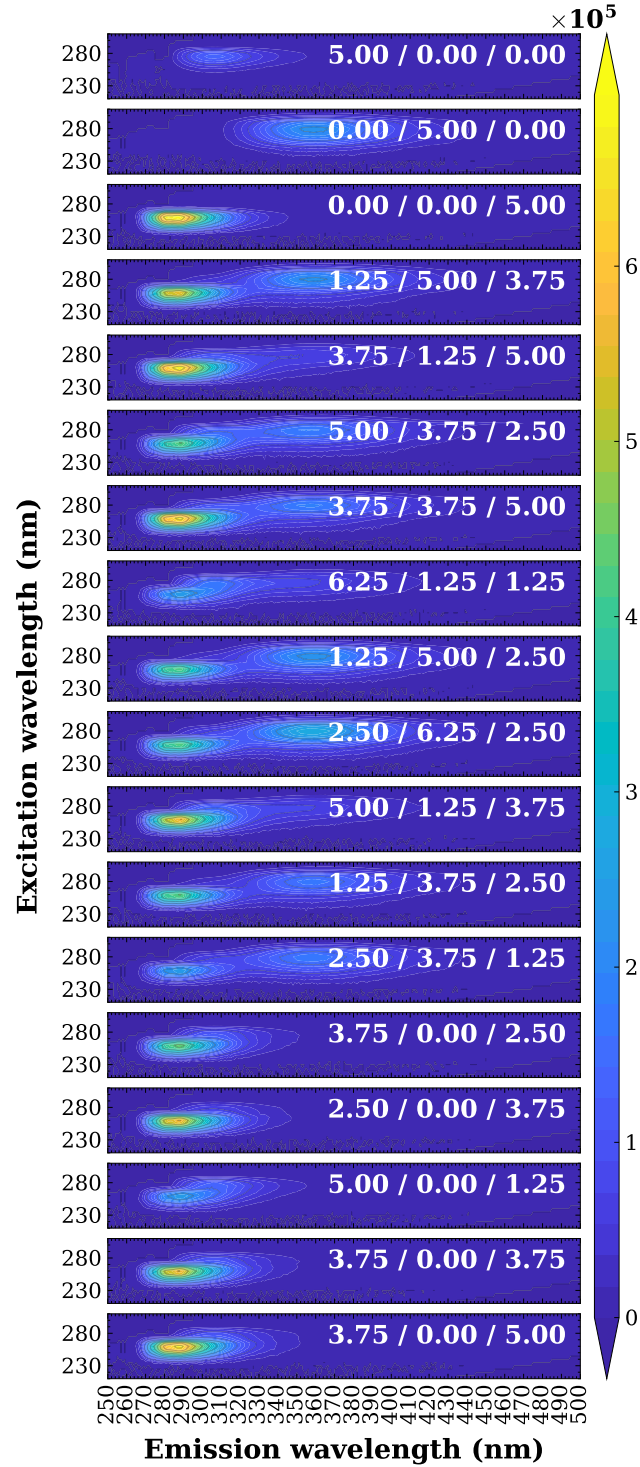


Figura 2.3: Profili d'intensità di emissione-eccitazione del tensore EEM. I profili corrispondono alle fette orizzontali del tensore, ordinate dall'alto ($T_{1,:,:}$) al basso ($T_{18,:,:}$). Visualizzazione di alcune matrici di eccitazione-emissione (EEM) per diversi campioni. Ogni profilo è etichettato a destra con le concentrazioni dei tre composti chimici (Val-Tyr-Val/Trp-Gly/Phe). Ogni profilo copre 21 lunghezze d'onda d'eccitazione (210, 215, ..., 310 nm) da 251 lunghezze d'onda di emissione (250, 251, ..., 500nm).

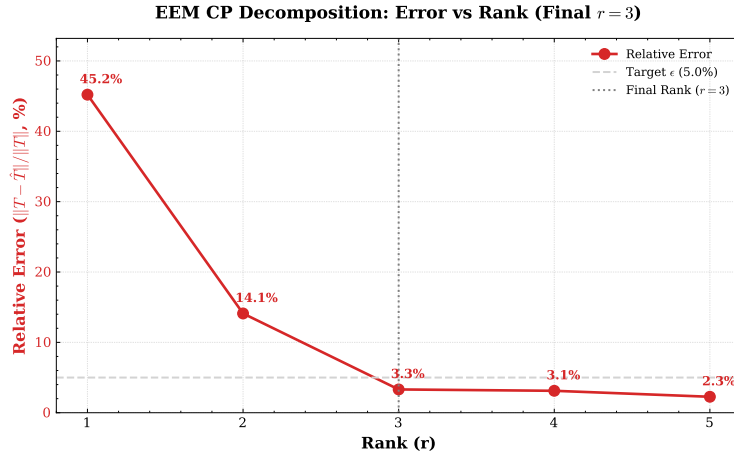


Figura 2.4: Errore relativo al variare del rango della decomposizione approssimante. Il rango ottimale selezionato è $r = 3$, corrispondente al superamento della tolleranza target $\varepsilon = 0.05$. Osserviamo che la precisione non varia di molto all'aumentare di r .

della decomposizione CP, dove ogni riga $f = 1, 2, 3$ rappresenta una sostanza diversa (scritta in sottoimpresione). Ad esempio, la componente a_1 sono le barre blu nella prima sottofigura in alto a sinistra, dove notiamo che i primi due fattori di a_1 sono nulli, che significa che la componente non è presente nei primi due campioni. Nella colonna centrale in alto denotiamo le componenti individuali di b_1 , che sono 256, con dei punti. Il terzo fattore c_1 è rappresentato nella figura in alto a destra.

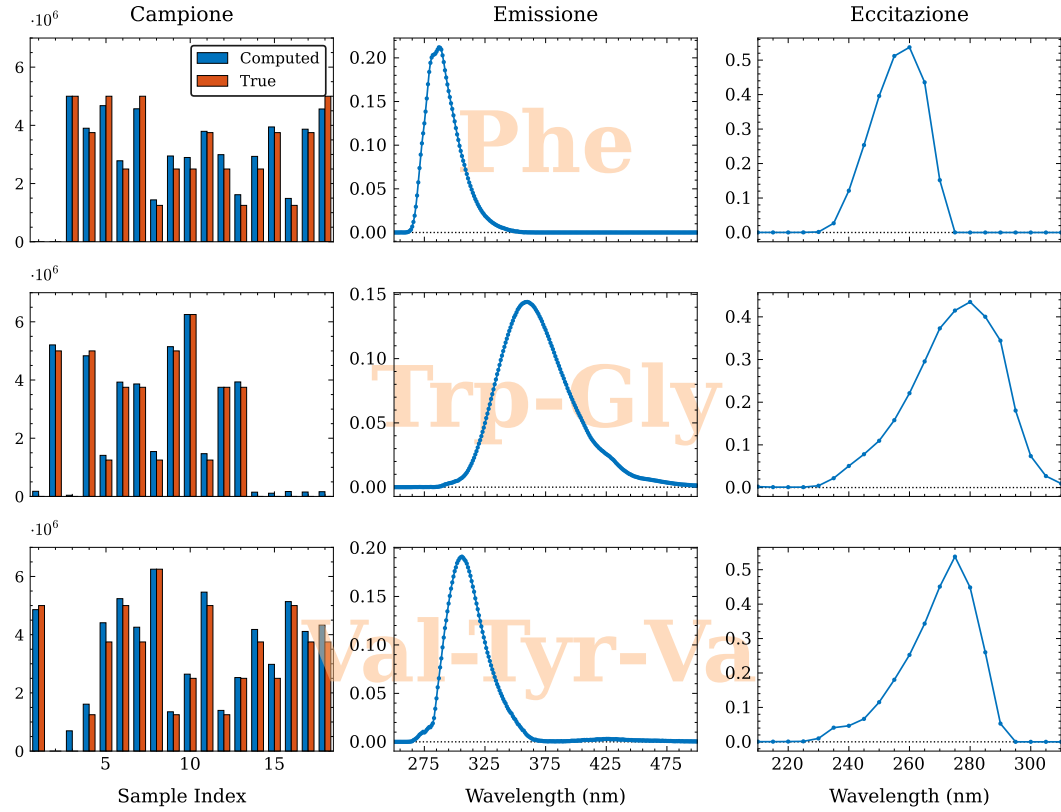


Figura 2.5: Fattori della decomposizione CP non negativa a rango $r = 3$ del tensore EEM. I diagrammi a barre a sinistra confrontano i profili di concentrazione calcolati con le concentrazioni effettive delle tre sostanze (Val-Tyr-Val, Trp-Gly, Phe).

2.2 BLIND DECONVOLUTION DI SEGNALI

Nel *problema del cocktail party* un insieme di persone si trova in una stanza a parlare. Ci sono diversi ricevitori impostati che registrano le conversazioni: non singolarmente ma tutte insieme, sovrapposte. L'obiettivo è recuperare cosa ogni persona sta dicendo. Questo “unmixing” si può ottenere usando la decomposizione CP. Questo problema è noto nell'analisi di segnali come *blind deconvolution*, utilizzata nel campo delle telecomunicazioni nel modello wireless *direct-sequence code-division multiple access* (DS-CDMA) [15, 24, 28] che riadattiamo in questa sezione.

Per semplicità supponiamo che R utenti mandino dei messaggi a I antenne. I messaggi arrivano mischiati, e l'obiettivo è recuperare i segnali individuali e le posizioni di utenti e antenne.

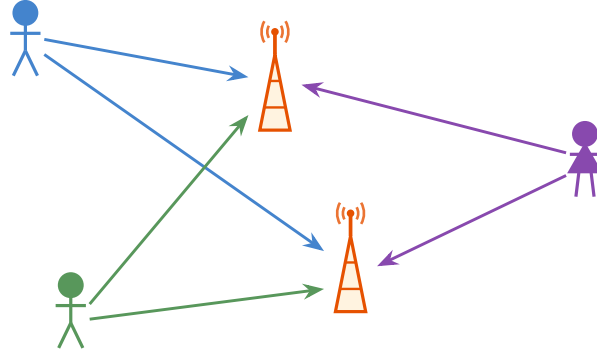


Figura 2.6: $R = 3$ sorgenti inviano segnali a $I = 2$ antenne.

L'utente r -esimo trasmette il messaggio in un vettore di $s_r = [s_{1r}, \dots, s_{Kr}]^T \in \mathbb{R}^K$, dove le entrate sono numeri reali random. Ci sono I antenne che ricevono i segnali. Quando i segnali arrivano all' i -esima antenna, il segnale è stato soggetto a distorsione nell'intensità e fase (anche detta *fading/gain*), quindi ciò che viene ricevuto dall'antenna i -esima sono le IK quantità $a_{ir}s_{kr}$, dove a_{ir} è il fading/gain tra l'utente r e l'antenna i .

In questo scenario si riceve un tensore (matrice) $T \in \mathbb{C}^{I \times K}$ di rango R di entrate

$$T_{ik} = \sum_{r=1}^R a_{ir}s_{kr}.$$

L'obiettivo sarebbe recuperare i segnali s_r , per $1 \leq r \leq R$ da questa matrice decomponendola come somma di matrici di rango uno. Sfortunatamente, questa decomposizione non è mai unica quando $R > 1$. Per aggirare il problema, vengono introdotti

dei “codici” per “diffondere” il segnale che ogni utente invia per ottenere un tensore a tre indici che avrà generalmente una decomposizione unica.¹

Facciamo le seguenti ipotesi sulla trasmissione dei segnali: la trasmissione avviene in linea d’aria, seguendo la strada più breve (secondo la distanza euclidea) senza incontrare ostacoli; i simboli inviati arrivano in maniera ordinata senza sovrapposizione (esempio $s_{k,r}$ allo stesso tempo di $s_{k+1,r}$). Questi comportamenti sono noti in letteratura come ICI (*interchip interference*) e ISI (*intersignal interference*). Inoltre i segnali non subiscono rumore e non vengono memorizzati.

Per recuperare s_r e a_{ir} , l’ r -esimo utente introduce una ridondanza in termini di codice chiamata *spreading sequence* che è fissata per ogni utente, diciamo $c_r = [c_{1r}, \dots, c_{Jr}]^\top$. Il simbolo s_{kr} è trasmesso su molteplici frequenze; come risultato, vengono trasmesse simultaneamente J quantità $[s_{kr}c_{1r}, \dots, s_{kr}c_{Jr}]$, chiamate *chips* al tempo k . La trasmissione totale sopra JK unità di tempo dell’utente r -esimo sono le JK chips $c_{jr}s_{kr}$, $1 \leq j \leq J$, $1 \leq k \leq K$ (qui J è anche detto *spreading gain*). L’antenna i -esima riceve le JK quantità $\sum_r a_{ir}c_{jr}s_{kr}$, per $1 \leq j \leq J$, $1 \leq k \leq K$, dove, come sopra, a_{ir} sono i fading/gain tra l’utente r e l’antenna i . In totale, viene ricevuto un tensore $T \in \mathbb{C}^{I \times J \times K}$ di rango R scrivibile in componenti come

$$(2.1) \quad T_{ijk} = \sum_{r=1}^R a_{ir}c_{jr}s_{kr},$$

o in generale,

$$T = \sum_{r=1}^R a_r \otimes c_r \otimes s_r.$$

In altre parole, $T = \llbracket S, C, A \rrbracket$. L’obiettivo è recuperare i segnali s_r , per $1 \leq r \leq R$ da questo tensore, cioè la matrice S . Per farlo abbiamo bisogno che la scrittura sia essenzialmente unica. Per il Teorema di Kruskal ?? questa decomposizione è unica se

$$\mathbf{k}_A + \mathbf{k}_B + \mathbf{k}_C \geq 2(R + 1).$$

Per costruzione [15] le matrici A e C sono di k -rango pieno con probabilità 1, mentre se i simboli trasmessi s_{Kr} appartengono ad un alfabeto finito, c’è possibilità che S non sia di k -rango pieno—c’è una possibilità che le sequenze di utenti diversi possano essere uguali, ma questo è poco probabile per dataset più grandi. Quindi nella pratica assumiamo che S sia di k -rango pieno. Questo significa che il numero

¹Tale spreading è comune nelle telecomunicazioni, e.g. della ridondanza è introdotta per compensare gli errori causati dalla corruzione dei segnali durante la trasmissione.

di utenti che possono essere processati simultaneamente si può considerare limitato come

$$(2.2) \quad \min\{I, R\} + \min\{J, R\} + \min\{K, R\} \geq 2(R + 1).$$

Il recupero di T avviene con l'algoritmo ALS

ref

Osservazione 2.2.1. Una limitazione di questo modello è che in ambito industriale spesso ci sono poche antenne e meno utenti, quindi si perde l'unicità.

ESPERIMENTO NUMERICO

Consideriamo lo scenario dove $R = 4$ utenti vogliono inviare messaggi di $K = 100$ simboli a $I = 5$ antenne, con un fattore di spreading $J = 16$. Generiamo dei tensori

$$T_\sigma = T_{\text{clean}} + W_\sigma \in \mathbb{R}^{I \times J \times K}$$

dove W_σ è un tensore di rumore gaussiano con entrate in $\mathcal{N}(0, \sigma)$ e T_{clean} è un tensore generato in formato denso a partire dalla decomposizione $T_{\text{clean}} = \llbracket A, C, S \rrbracket$. L'esperimento consiste in recuperare la posizione degli r utenti conoscendo la posizione delle antenne (la matrice A). Per farlo dobbiamo calcolare le matrici S e C , sfruttando la decomposizione CP al variare del rumore σ nella Tabella 2.1.

Tabella 2.1: Livelli di rumore gaussiano applicati al tensore $T_\sigma = T_{\text{clean}} + W_\sigma$, con relativo rapporto segnale-rumore (SNR) e corrispondente scenario fisico di trasmissione.

Rumore	σ	SNR _{dB}	Scenario fisico
σ_0	0,0000	∞ dB	Assenza di rumore
σ_1	0,0173	16,5 dB	Segnale ottimo
σ_2	0,0403	9,1 dB	Segnale buono
σ_3	0,0691	4,4 dB	Segnale debole
σ_4	0,1037	0,9 dB	Segnale quasi assente
σ_5	0,1382	-1,6 dB	Fortissime interferenze

- Matrice delle antenne $A \in \mathbb{R}^{I \times R}$: generiamo le posizioni delle antenne p_i e degli utenti u_r nel quadrato $[0, L]^2$, dove L è un generico intero positivo, per $i = 1, \dots, I$ e $r = 1, \dots, R$. Le entrate di A sono $a_{ir} := \frac{1}{\|p_i - u_r\|_2}$, dove assumiamo che $\|p_i - u_r\|_2 > 0$ (gli utenti non si trovano sulle antenne).
- Matrice dei simboli $S \in \mathbb{R}^{K \times R}$ con entrate s_{kr} secondo la distribuzione gaussiana standard $\mathcal{N}(0, 1)$.

- Matrice delle sequenze di spreading $C \in \{-1, 1\}^{J \times R}$: contiene i codici casuali assegnati a ciascuna delle R sorgenti, dove $c_{jr} = -1$ o $c_{jr} = 1$ con uguale probabilità.

ref

L'algoritmo ALS produce un tensore approssimante $\hat{T} = \llbracket \hat{A}, \hat{C}, \hat{S} \rrbracket$ unico a meno di permutazione e riscalamento dei fattori: per costruzione, l'Equazione 2.2 è verificata in senso stretto. In questa applicazione è importante recuperare l'ordine dei fattori—altrimenti un utente finirebbe per trovarsi sulla posizione di un altro utente. Descriviamo brevemente il procedimento. Indichiamo con (a_r, c_r, s_r) i fattori di T e con $(\hat{a}_r, \hat{c}_r, \hat{s}_r)$ i fattori di $\hat{T}$.

1. Calcoliamo correlazione tra ogni colonna $\hat{a}_k$ e a_r tramite $\rho_{kr} := \frac{\hat{a}_k^\top a_r}{\|\hat{a}_k\|_2 \|a_r\|_2}$.
2. Costruiamo la matrice dei costi $M \in \mathbb{R}^{R \times R}$ di entrate $M_{kr} = 1 - |\rho_{kr}|$.
3. Troviamo una permutazione ottima $\pi^* \in \mathfrak{S}_R$ che realizza

$$\min_{\pi \in \mathfrak{S}_R} \sum_{r=1}^R M_{\pi(r), r}.$$

4. Calcoliamo il cambio di segno tramite $\xi_r := \text{sign}(\hat{a}_{\pi^*(r)}^\top a_r) \in \{-1, 1\}$.
5. Permutiamo e riscaliamo $\hat{A}$ e $\hat{S}$ applicando $\xi_r \hat{A}_{:, \pi^*(r)}$ e $\xi_r \hat{S}_{:, \pi^*(r)}$, rispettivamente.

RISULTATI

Nel contesto definito sopra, denotiamo gli utenti reali $r \in \{1, \dots, R\}$ con dei quadrati e gli utenti recuperati con degli omini dello stesso colore. Inizialmente verifichiamo che in assenza di rumore l'utente recuperato combaci con l'utente reale, come illustrato in Figura 2.7. Inoltre verifichiamo che ciò funzioni per diverse disposizioni degli utenti e delle antenne in Figura 2.8. Di seguito, generiamo più tensori T_σ al variare di σ nella Tabella 2.1 e osserviamo quanto gli utenti recuperati si discostano dall'utente originale. In Figura 2.9 osserviamo i comportamenti tipici di deriva al crescere del rumore. Infine, la Figura 2.10 mostra ulteriori esecuzioni dell'algoritmo su sei configurazioni spaziali differenti.

Cosa ne pensa di queste figure?

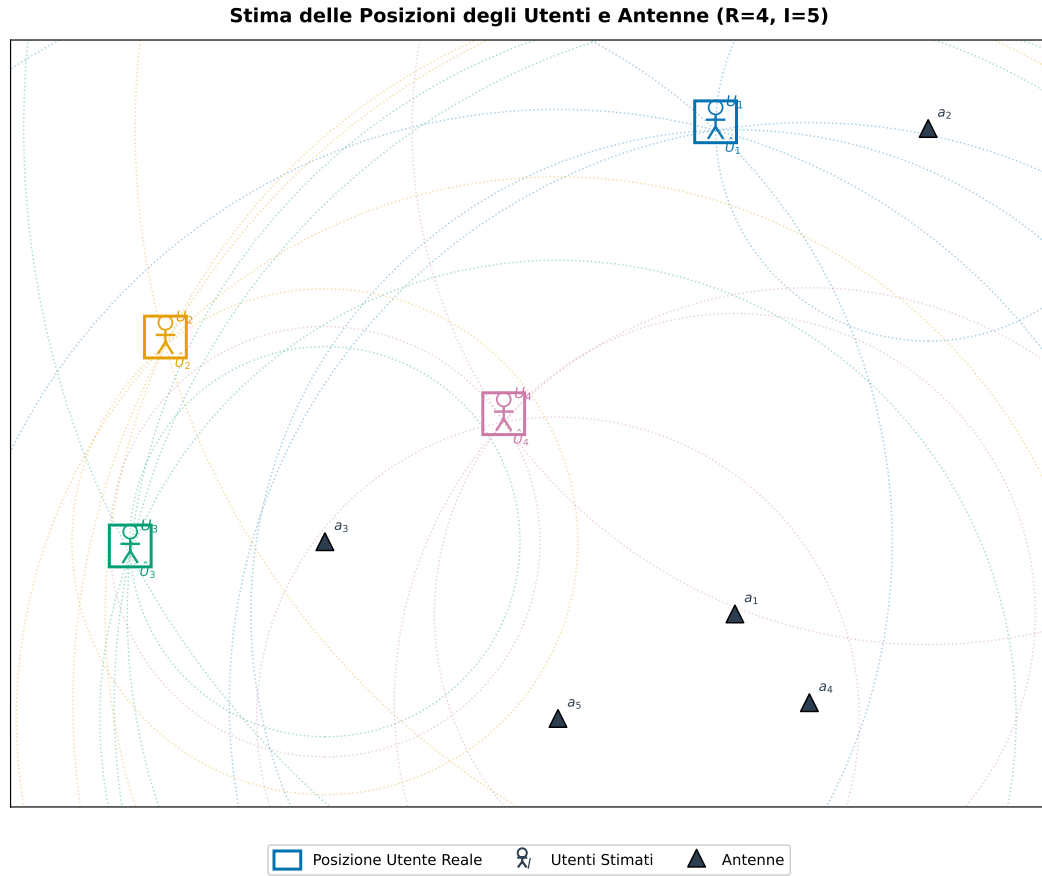


Figura 2.7: Stima della posizione degli utenti e delle antenne in assenza di rumore. I quadrati trasparenti indicano le posizioni reali degli utenti U_r , gli omini indicano le posizioni stimate $\hat{U}_r$, i triangoli scuri indicano le antenne a_i , e le circonferenze tratteggiate indicano le distanze stimate dalle antenne.

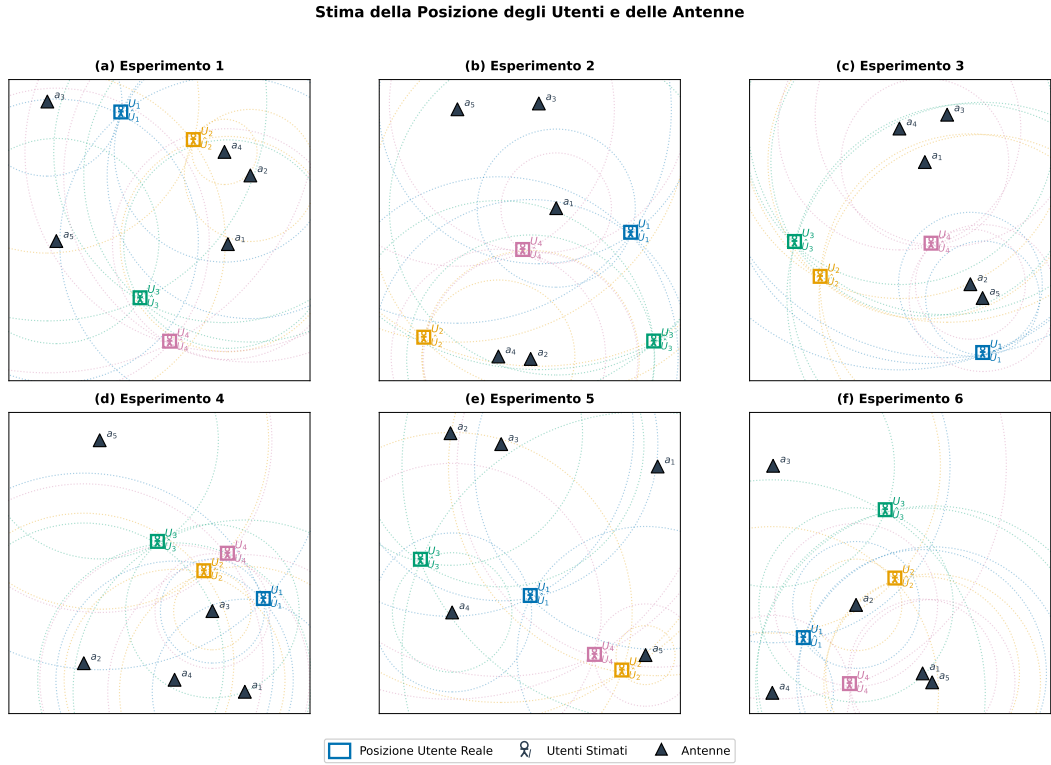


Figura 2.8: Stima della posizione degli utenti e delle antenne in sei esperimenti indipendenti con diverse disposizioni spaziali di antenne e sorgenti in assenza di rumore.

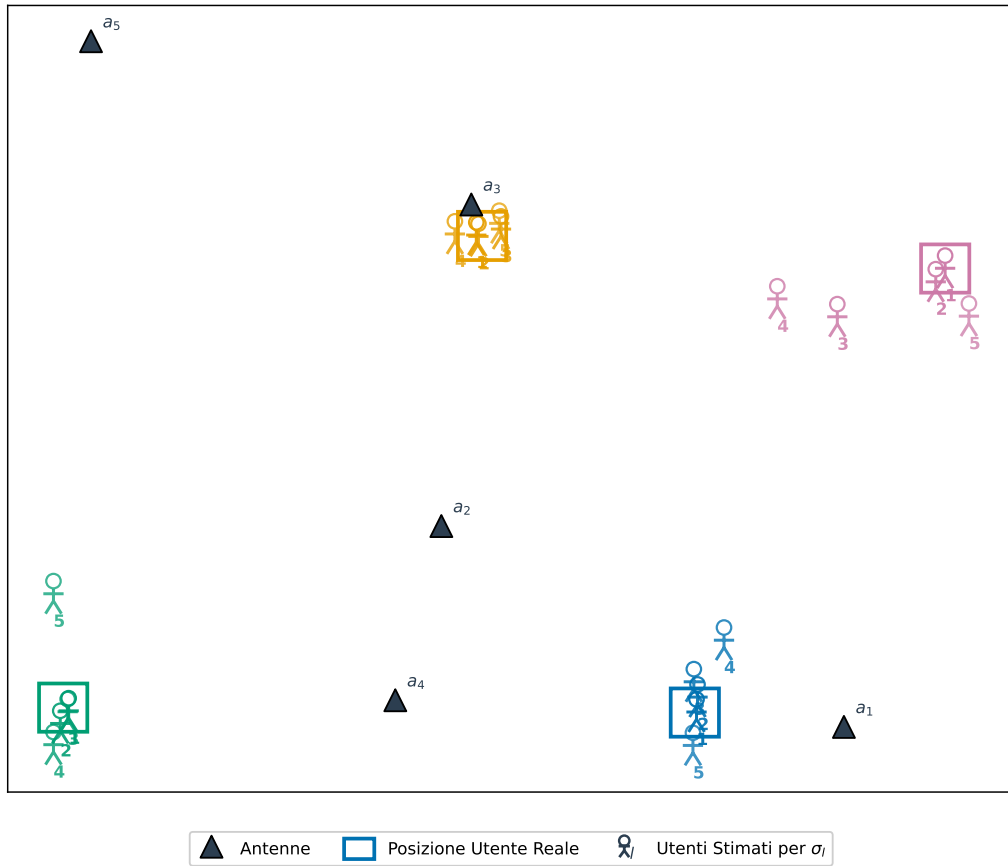
Deriva della Localizzazione degli Utenti sotto Rumore Gaussiano ($R=4$, $I=5$)

Figura 2.9: Deriva della localizzazione degli utenti sotto sei livelli crescenti di rumore gaussiano $(\sigma_0, \dots, \sigma_5)$ per un singolo esperimento. Gli omini numerati da 1 a 5 indicano le posizioni stimate al crescere del livello di rumore.



Figura 2.10: Deriva della localizzazione degli utenti sotto rumore gaussiano per sei esperimenti indipendenti (disposizioni spaziali differenti), mostrando i comportamenti tipici di deriva al crescere del livello di rumore σ_l .

ACRONIMI

ALS	Alternating Least Squares
APA	Arbitrary Precision Approximating
CP	Canonical Polyadic Decomposition
SVD	Singular Value Decomposition

GLOSSARIO

L ^A T _E X	A document preparation system
$\mathbb{R}$	The set of real numbers

BIBLIOGRAFIA

1. E. Acar, E. E. Papalexakis, G. Gürdeniz, M. A. Rasmussen, A. J. Lawaetz, M. Nilsson, e R. Bro. «Structure-Revealing Data Fusion». *BMC Bioinformatics* 15:1, 2014, p. 239. DOI: [10.1186/1471-2105-15-239](https://doi.org/10.1186/1471-2105-15-239).
2. J. Alman, R. Duan, V. V. Williams, Y. Xu, Z. Xu, e R. Zhou. «More asymmetry yields faster matrix multiplication». *Computing Research Repository (arXiv)*, 2024. arXiv:2404.16349.
3. J. Alman, V. Vassilevska Williams et al. «Improving the matrix multiplication exponent with modern optimization and AlphaEvolve». *arXiv preprint*, 2026.
4. G. Ballard, A. R. Benson, A. Druinsky, B. Lipshitz, e O. Schwartz. «Improving the numerical stability of fast matrix multiplication». *SIAM Journal on Matrix Analysis and Applications* 37:4, 2016, pp. 1382–1418.
5. G. Ballard e T. G. Kolda. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025. URL: <https://tensortextbook.com>.
6. A. R. Benson e G. Ballard. «A Framework for Practical Parallel Fast Matrix Multiplication». *CoRR* abs/1409.2908, 2014. arXiv: [1409.2908](https://arxiv.org/abs/1409.2908). URL: <http://arxiv.org/abs/1409.2908>.
7. D. Bini. «Relations between exact and approximate bilinear algorithms. Applications». *Calcolo* 17:1, 1980, pp. 87–97. DOI: [10.1007/bf02575865](https://doi.org/10.1007/bf02575865). URL: <https://doi.org/10.1007/bf02575865>.
8. D. Bini e G. Lotti. «Stability of fast algorithms for matrix multiplication». *Numerische Mathematik* 36:1, 1980, pp. 63–72.
9. M. Bläser. «A $(5/2)n^2$ -Lower Bound for the Multiplicative Complexity of $n \times n$ -Matrix Multiplication». In: *Proceedings of the 18th Annual Symposium on Theoretical Aspects of Computer Science*. STACS '01. Springer-Verlag, Berlin, Heidelberg, 2001, pp. 99–109. ISBN: 3540416951.

10. M. Bläser. «On the complexity of the multiplication of matrices of small formats». *Journal of Complexity* 19:1, 2003, pp. 43–60. ISSN: 0885-064X. DOI: [https://doi.org/10.1016/S0885-064X\(02\)00007-9](https://doi.org/10.1016/S0885-064X(02)00007-9). URL: <https://www.sciencedirect.com/science/article/pii/S0885064X02000079>.
11. P. Bürgisser, M. Clausen, e M. A. Shokrollahi. *Algebraic complexity theory*. Vol. 315. Springer Science & Business Media, 2013.
12. D. Coppersmith e S. Winograd. «Matrix multiplication via arithmetic progressions». *Journal of Symbolic Computation* 9:3, 1990. Computational algebraic complexity editorial, pp. 251–280. ISSN: 0747-7171. DOI: [https://doi.org/10.1016/S0747-7171\(08\)80013-2](https://doi.org/10.1016/S0747-7171(08)80013-2). URL: <https://www.sciencedirect.com/science/article/pii/S0747717108800132>.
13. T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein. *Introduction to algorithms*. MIT press, 2022.
14. P. D’Alberto. *Strassen’s Matrix Multiplication Algorithm Is Still Faster*. 2023. arXiv: [2312.12732](https://arxiv.org/abs/2312.12732) [cs.MS]. URL: <https://arxiv.org/abs/2312.12732>.
15. L. DeLathauwer e A. de Baynast. «Blind deconvolution of DS-CDMA signals by means of decomposition in rank-(1, L, L) terms». *IEEE Transactions on Signal Processing* 56:4, 2008, pp. 1562–1571.
16. A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz et al. «Discovering faster matrix multiplication algorithms with reinforcement learning». *Nature* 610:7930, 2022, pp. 47–53.
17. F. L. Gall. «Powers of Tensors and Fast Matrix Multiplication». *CoRR* abs/1401.7714, 2014. arXiv: [1401.7714](https://arxiv.org/abs/1401.7714). URL: <http://arxiv.org/abs/1401.7714>.
18. J. D. Garrett. «garrettj403/SciencePlots». Ver. 1.0.9, 2021. DOI: [10.5281/zenodo.4106649](https://doi.org/10.5281/zenodo.4106649). URL: <http://doi.org/10.5281/zenodo.4106649>.
19. Intel Corporation. *Intel® oneAPI Math Kernel Library Reference Manual*. Intel Corporation. Santa Clara, CA, 2024. URL: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html>.
20. T. G. Kolda. *EEM Tensor Dataset*. https://gitlab.com/tensors/tensor_data_eem. Accesso: 25 agosto 2026. 2021.

21. J. B. Kruskal. «Three-way arrays: rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics». *Linear Algebra and its Applications* 18:2, 1977, pp. 95–138. ISSN: 0024-3795. DOI: [https://doi.org/10.1016/0024-3795\(77\)90069-6](https://doi.org/10.1016/0024-3795(77)90069-6). URL: <https://www.sciencedirect.com/science/article/pii/0024379577900696>.
22. J. D. Laderman. «A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications». *Bulletin of the American Mathematical Society* 82:1, 1976, pp. 126–128.
23. J. M. Landsberg. «Geometry and complexity theory». *NSF Award Number 1405348. Directorate for Mathematical and Physical Sciences* 14:1405348, 2015, p. 5348.
24. J. M. Landsberg. *Tensors: geometry and applications*. Vol. 128. American Mathematical Soc., 2011.
25. W. Miller. «Computational complexity and numerical stability». In: *Proceedings of the sixth annual ACM symposium on Theory of computing*. 1974, pp. 317–322.
26. V. Y. Pan. «Strassen’s algorithm is not optimal trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations». In: *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*. 1978, pp. 166–176. DOI: [10.1109/SFCS.1978.34](https://doi.org/10.1109/SFCS.1978.34).
27. A. Schönhage. «Partial and Total Matrix Multiplication». *SIAM Journal on Computing* 10:3, 1981, pp. 434–455. DOI: [10.1137/0210032](https://doi.org/10.1137/0210032). eprint: <https://doi.org/10.1137/0210032>. URL: <https://doi.org/10.1137/0210032>.
28. N. D. Sidiropoulos, G. B. Giannakis, e R. Bro. «Blind PARAFAC receivers for DS-CDMA systems». *IEEE Transactions on Signal Processing* 48:3, 2000, pp. 810–823.
29. A. K. Smilde, R. Bro, P. Geladi et al. *Multi-way analysis with applications in the chemical sciences*. Vol. 396. Wiley Chichester, 2004.
30. A. Stothers. «On the complexity of matrix multiplication». Tesi di dott. Edinburgh, UK: University of Edinburgh, 2010. URL: <https://ed.ac.uk>.

31. V. STRASSEN. «Relative bilinear complexity and matrix multiplication.» *Journal für die reine und angewandte Mathematik* 1987:375-376, 1987, pp. 406–443. DOI: [doi:10.1515/crll.1987.375-376.406](https://doi.org/10.1515/crll.1987.375-376.406). URL: <https://doi.org/10.1515/crll.1987.375-376.406>.
32. V. Strassen. «Gaussian elimination is not optimal». *Numer. Math.* 13:4, 1969, pp. 354–356. ISSN: 0029-599X. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411). URL: <https://doi.org/10.1007/BF02165411>.
33. V. V. Williams. *Breaking the Coppersmith-Winograd barrier*. Manuscript available at <http://theory.stanford.edu/~virgi/matrixmult-f.pdf>. 2011.
34. S. Winograd. «On multiplication of 2×2 matrices». *Linear Algebra and its Applications* 4:4, 1971, pp. 381–388. ISSN: 0024-3795. DOI: [https://doi.org/10.1016/0024-3795\(71\)90009-7](https://doi.org/10.1016/0024-3795(71)90009-7). URL: <https://www.sciencedirect.com/science/article/pii/0024379571900097>.